

Chapter 01: From a Question to Ordinary Least Squares

A zero-prerequisite guide to geometry, calculus, geography, and implementation

Revised edition — August 2026

Contents

Chapter 1 — From a Question to Ordinary Least Squares	3
Level 1 Data Explorer: Week 1 of the MHP Cost Estimator	3
The pedagogical contract	3
Week 1 learning outcomes	4
The five-day route	4
Running case and units	4
Day 1 — What Question Are We Asking?	5
1.1 Regression in one sentence	5
1.2 Prediction: estimating an unknown outcome	6
1.3 Explanation: describing conditional association	6
1.4 Causation: estimating an intervention	8
1.5 Confounding: the hidden common cause	8
1.6 Simpson’s paradox: when aggregation reverses a pattern	9
1.7 A decision protocol before touching code	10
1.8 Build: start the application with a question contract	11
1.9 Break it deliberately	12
1.10 Day 1 practice	12
Day 2 — Regression Vocabulary and the Shape of Data	13
2.1 The vocabulary of one project table	13
2.2 Scalars, vectors, matrices, and tensors	13
2.3 Shape is part of meaning	14
2.4 The design matrix and row alignment	14
2.5 A data dictionary is part of the model	15
2.6 Parsing one nested project record	15
2.7 Code proof: matrix-vector multiplication by loops	16
2.8 Build: robust input validation	16
2.9 Break it deliberately	17
2.10 Day 2 practice	18
Day 3 — Linear Algebra as a System of Predictions	18
3.0 Matplotlib in five lines	18
3.0A Visual mathematics bridge: from a map to a graph	19
3.0B Slope: rise divided by run	19
3.0C Intercept: where the line meets an axis	22
3.0D Curves, secants, and tangents	22
3.0E Geography is more than a point on a map	24
3.1 From a verbal rule to a linear equation	24

3.2 Multiple features are still a weighted sum	25
3.3 Why a column of ones creates an intercept	25
3.4 Code proof: expand $x @ \text{beta}$ by hand	25
3.5 Shape algebra before numerical algebra	26
3.6 Matrices as transformations—and the limit of the analogy	26
3.7 Figure lab: with and without an intercept	27
3.8 Scaling and units	28
3.9 Build: intercept construction and prediction	28
3.10 Break it deliberately: broadcasting is not matrix multiplication	29
3.11 Day 3 practice	29
Day 4 — Residuals, Loss, and the Geometry of OLS	30
4.1 A prediction becomes testable through its residual	30
4.2 Why OLS squares residuals	31
4.3 SSR, MSE, RMSE, and MAE answer different reporting needs	31
4.4 OLS as a constrained search in observation space	32
4.5 The Pythagorean proof of minimum distance	33
4.6 Why orthogonality produces the normal equations	34
4.7 Code proof: orthogonality and Pythagoras	34
4.8 Figure lab: draw the projection in three dimensions	35
4.9 Build: metrics as methods	36
4.10 Break it deliberately: one outlier	36
4.11 Day 4 practice	36
Day 5 — Deriving, Fitting, and Stress-Testing OLS	37
5.0 Calculus bridge: change, tangent, derivative, and minimum	37
5.1 The scalar error bowl	41
5.2 One feature with an intercept	45
5.3 The matrix objective	48
5.4 The matrix derivative without hand-waving	49
5.5 Isolating $\hat{\beta}$ and stating the missing condition	51
5.6 Why we do not compute the inverse in application code	52
5.7 Rank failure: two columns carrying the same information	53
5.8 Figure lab: see the error valley	54
5.9 Build: the complete Week 1 estimator	55
5.10 End-to-end use on synthetic KP projects	58
5.11 Verify the class against the equations	59
5.12 What the completed application still cannot do	59
5.13 Concept checkpoint: explain before coding	59
5.14 Day 5 practice	60
Week 1 Capstone — Build It, Break It, Rebuild It	60
Capstone brief	60
Assessment rubric	61
Formula Sheet	62
Glossary	62
Instructor and Self-Study Notes	63
Suggested daily timebox	63
Retrieval prompts for the next morning	63
Guidance on the KP context	63
Where Week 2 Begins	64
Final reflection	64

Chapter 1 — From a Question to Ordinary Least Squares

Level 1 Data Explorer: Week 1 of the MHP Cost Estimator

Central promise. By the end of this chapter, you will not merely be able to call a regression function. You will be able to state the question a regression can answer, represent the data correctly, derive ordinary least squares (OLS) in scalar and matrix form, implement it without a machine-learning library, diagnose when it fails, and explain the result to a decision-maker responsible for public infrastructure.

No-prerequisite promise. This chapter assumes no previous knowledge of machine learning, calculus, or geometry. Words such as axis, coordinate, slope, tangent, derivative, gradient, and projection are built from ordinary ideas before they are used in formulas. If a later section feels too compressed, return to the visual mathematics bridge in Day 3 and the calculus bridge at the beginning of Day 5.

This chapter uses a running case: estimating the cost of microhydro power (MHP) projects in Khyber Pakhtunkhwa (KP). The projects are fictional but realistic. Their geography, procurement problems, transport constraints, and management decisions are grounded in the kinds of conditions infrastructure teams encounter in Chitral, Dir, Swat, and other mountainous districts.

A note if you just finished Chapter 0. This is a new dataset, not a continuation of the development, agricultural, or infrastructure CSVs you generated there. Those built the reflex; this is where it gets used. The MHP case starts small — eight projects below — and grows with each chapter.

The case is deliberately high-stakes. A regression coefficient is not just a number on a screen. If misunderstood, it can lead to an unrealistic budget, an unfair judgement about a field team, or a decision to withdraw a service from a remote community.

The pedagogical contract

This book asks the learner and the author to keep the same contract.

1. **One central concept per day.** Each lesson has a single intellectual centre. Supporting ideas are introduced only when that centre requires them.
2. **No mathematical hand-waving.** A displayed equation is unpacked symbol by symbol and connected to numbers.
3. **Code as proof.** Important equations are verified with NumPy and, where useful, with plain loops. We do not use scikit-learn in Week 1.
4. **Geometry before memorisation.** Vectors, projections, residuals, and error surfaces are drawn or plotted.
5. **Build, break, rebuild.** Every day adds to the `MHPCostEstimator`, then deliberately creates a failure that the learner must diagnose.
6. **Interpretation in context.** Every mathematical result is translated back into a budget, engineering, or management statement.
7. **Cognitive load is managed, not intelligence underestimated.** New notation is introduced gradually, but no essential reasoning is hidden.

What “from scratch” means here

We will use Python, NumPy, and Matplotlib as basic tools. We will not ask a library to fit a regression model for us. NumPy stores arrays and performs arithmetic; the learner still

constructs the design matrix, objective function, normal equations, parameter estimates, predictions, residuals, and diagnostics.

Later chapters can remove even more scaffolding—for example, by implementing matrix decomposition manually—but doing that now would bury the statistical idea under numerical bookkeeping.

Week 1 learning outcomes

At the end of five days, you should be able to:

- distinguish prediction, explanation, and causal inference;
- identify a target, features, observations, parameters, and hyperparameters;
- read and verify the shapes of vectors and matrices;
- explain why a column of ones creates an intercept;
- calculate residuals, SSR, MSE, RMSE, and MAE by hand and in code;
- describe OLS as both error minimisation and orthogonal projection;
- derive the scalar and matrix OLS solutions;
- explain when the OLS solution is unique;
- detect perfect multicollinearity and other common failures;
- fit and evaluate a small regression model without a machine-learning library; and
- separate a mathematically correct calculation from a defensible policy conclusion.

The five-day route

Each day follows the same rhythm:

1. **Orient:** connect the concept to a decision;
2. **Construct:** develop the mathematics carefully;
3. **Prove:** test the mathematics with raw code;
4. **See:** create a diagram or figure;
5. **Build:** add one working part to the application;
6. **Break:** produce a predictable failure;
7. **Reflect:** decide what the result does and does not justify.

Running case and units

The MHP Cost Estimator will eventually use multiple project features. During the first four days, we also use very small datasets so that every number can be inspected.

Field	Symbol	Unit	Meaning
Project count	n	projects	Number of observations
Feature count	p	features	Number of explanatory columns before the intercept
Cable length	x_1	km	Length of distribution cable
Hydraulic head	x_2	m	Vertical fall available to drive the turbine
Road distance	x_3	km	Distance from an all-weather road to the site

Field	Symbol	Unit	Meaning
Terrain difficulty	x_4	index 1–5	Ordered field assessment of access difficulty
Planned capacity	x_5	kW	Intended generating capacity
Actual project cost	y	million PKR	Target to be estimated

Unit discipline: A coefficient is meaningless without its units. If cost is recorded in million PKR and cable length in kilometres, the cable coefficient has units of “million PKR per kilometre,” holding the other included features constant.

The data in this chapter are **synthetic**. They are for learning, not for judging a real district, contractor, community, or organisation.

Minimal software setup

The code assumes Python 3.11 or later. In a terminal, create an isolated environment and install the two numerical packages used in this chapter:

```
python -m venv .venv
source .venv/bin/activate          # Windows PowerShell: .venv\Scripts\Activate.ps1
python -m pip install numpy matplotlib
```

Use a plain .py file, a Jupyter notebook, or an editor with an interactive Python terminal. Keep every day’s experiments. A failed calculation with a short note about why it failed is part of the learning record, not clutter to erase.

Day 1 — What Question Are We Asking?

Today’s central idea: Prediction, explanation, and causation are different jobs. A regression calculation does not decide which job you meant.

1.1 Regression in one sentence

Regression constructs a numerical relationship between a target variable y and one or more observed features X .

That sentence describes the mechanism but not the purpose. The same regression equation can be used for three very different purposes.

Job	Question	What success looks like	Main danger
Prediction	“How accurately can we estimate the cost of a new project?”	Low error on genuinely new projects	A model may fail when conditions change

Job	Question	What success looks like	Main danger
Explanation	“How is road distance associated with cost after accounting for included features?”	Stable, interpretable coefficients with uncertainty	Omitted variables and correlated features distort interpretation
Causal inference	“How much would cost change if we improved road access?”	Credible estimate of an intervention	Association is mistaken for the effect of an action

The distinction is not philosophical decoration. It changes the data you need, the assumptions you must defend, and the way the result may be used.

1.2 Prediction: estimating an unknown outcome

For prediction, we want a function f that converts the observed features of project i into an estimate:

$$\hat{y}_i = f(x_{i1}, x_{i2}, \dots, x_{ip}).$$

Read the symbols slowly:

- i identifies one project;
- x_{ij} is feature j for project i ;
- f is the fitted prediction rule;
- y_i is the actual cost; and
- $\hat{y}_i$, pronounced “y-hat,” is the estimated cost.

A predictor may be useful even when a feature is not causal. For instance, the number of difficult river crossings may predict transport cost because it summarises remoteness. A planning unit can use that signal to improve a contingency estimate without claiming that a river crossing, by itself, causes every extra rupee.

Prediction must be evaluated on data not used to fit the model. A rule that memorises completed projects can appear perfect and still fail on the next valley.

1.3 Explanation: describing conditional association

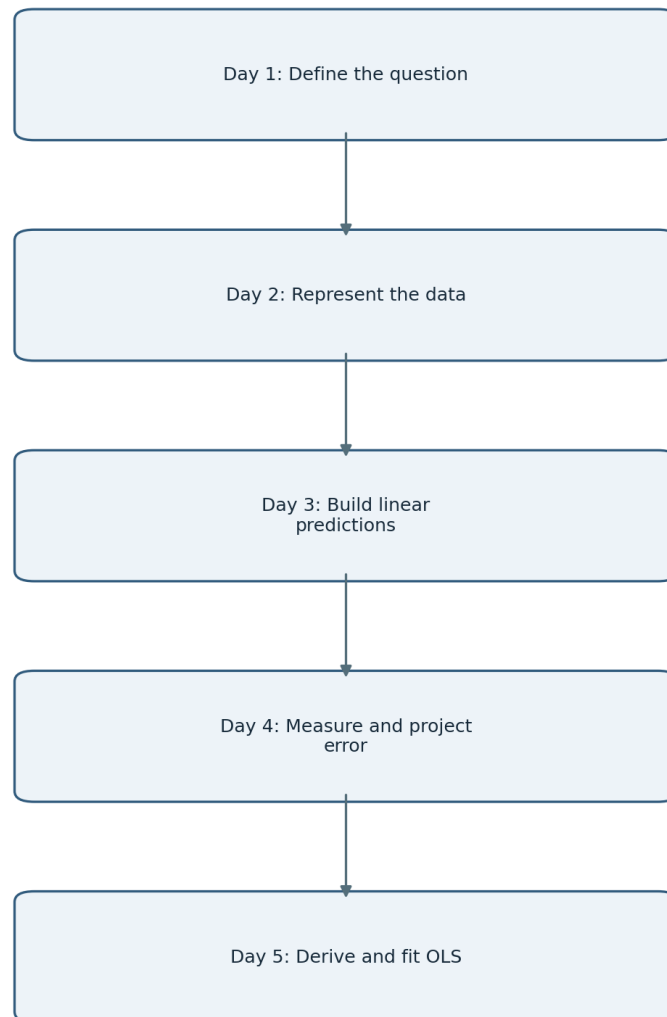
For explanation, attention shifts from the prediction alone to the model’s parameters. Consider:

$$\hat{y}_i = \beta_0 + \beta_1 \text{road_distance}_i + \beta_2 \text{capacity}_i.$$

Here β_1 describes the estimated difference in cost associated with one additional kilometre of road distance **among projects with the same included capacity**.

The phrase in bold is essential. In multiple regression, a coefficient is conditional on the other included features. It does not automatically control for variables that are absent, poorly measured, or structurally different across districts.

An explanatory analysis also needs uncertainty. A coefficient of 0.8 million PKR per kilometre is not equally persuasive if it comes from 6 projects or 600 projects. Standard errors and confidence intervals arrive later; Week 1 first establishes what is being estimated.

**Figure 1:** Concept diagram

1.4 Causation: estimating an intervention

A causal question asks about a change we could make:

If an all-weather access road were constructed before the MHP works began, how would the final project cost differ from what it would otherwise have been?

This question compares two potential outcomes for the same project:

$$\text{causal effect}_i = Y_i(\text{road improved}) - Y_i(\text{road not improved}).$$

We can observe only one of those outcomes for a given project. The unobserved alternative is the **counterfactual**. Causal inference is therefore a problem of constructing a credible comparison, not merely fitting a line.

Ordinary regression can be part of a causal design, but the regression output is causal only when the research design and assumptions make it so. Random assignment, natural experiments, careful matching, panel designs, and defensible causal graphs can help. A high R^2 cannot turn an associational design into a causal one.

1.5 Confounding: the hidden common cause

Suppose projects in rugged terrain are more likely to require helicopter transport, and rugged terrain independently increases excavation delays and labour costs.

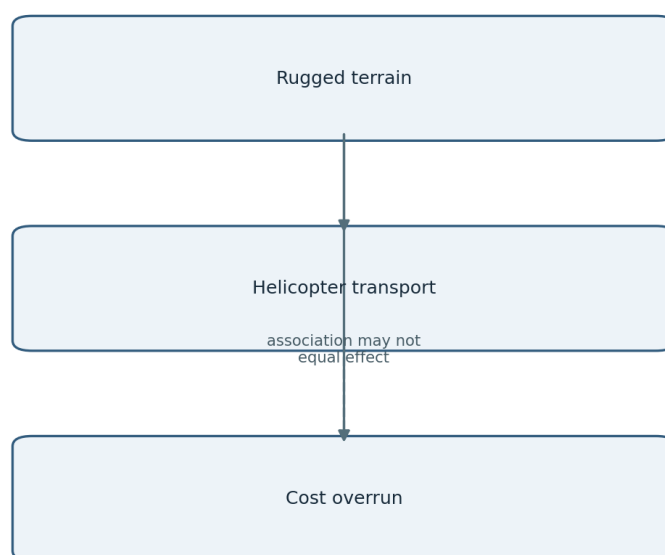


Figure 2: Concept diagram

Figure 1.1 — A confounding structure. Terrain is a common cause of the selected transport method and the cost overrun. The dotted arrow is the relationship we might estimate naively.

If terrain is ignored, projects using helicopters will look more expensive even if helicopter transport reduced cost relative to the feasible alternative—weeks of manual carriage or an unfinished project.

A variable is a confounder when it is causally related to both the exposure or decision and the

outcome, and is not itself a consequence of that exposure. The precise definition depends on the full causal structure; “control for everything available” is not a safe rule because controlling for mediators or colliders can introduce new bias.

1.6 Simpson’s paradox: when aggregation reverses a pattern

Imagine eight projects, four in relatively accessible terrain and four in remote mountain terrain. Within each terrain group, longer cable raises cost. Yet remote projects happen to use shorter cable and start from a much higher cost base.

Terrain group	Cable length (km)	Cost (million PKR)
Accessible	4	18
Accessible	5	20
Accessible	6	22
Accessible	7	24
Remote	1	30
Remote	2	33
Remote	3	36
Remote	4	39

Within accessible projects, one more kilometre is associated with about 2 million PKR more cost. Within remote projects, it is associated with about 3 million PKR more cost. But in the combined data, low cable lengths are concentrated among costly remote projects, so the overall line slopes downward.

For one feature, the OLS slope can be written as:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}.$$

The numerator measures how x and y move together. The denominator measures how much x varies. Day 5 derives this formula; today we use it as a diagnostic.

Code proof: calculate all three slopes

```
import numpy as np

def slope_from_sums(x, y):
    """One-feature OLS slope, written directly from the equation."""
    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)
    x_centered = x - x.mean()
    y_centered = y - y.mean()
    numerator = np.sum(x_centered * y_centered)
    denominator = np.sum(x_centered ** 2)
    return numerator / denominator

x_accessible = np.array([4, 5, 6, 7], dtype=float)
y_accessible = np.array([18, 20, 22, 24], dtype=float)

x_remote = np.array([1, 2, 3, 4], dtype=float)
y_remote = np.array([30, 33, 36, 39], dtype=float)

x_all = np.concatenate([x_accessible, x_remote])
y_all = np.concatenate([y_accessible, y_remote])
```

```
print("Accessible slope:", slope_from_sums(x_accessible, y_accessible))
print("Remote slope:   ", slope_from_sums(x_remote, y_remote))
print("Combined slope: ", slope_from_sums(x_all, y_all))

# Expected, approximately:
# Accessible slope:  2.0
# Remote slope:      3.0
# Combined slope:    -2.0
```

The code does not “prove” which relationship is causal. It proves the numerical reversal and forces us to explain it using geography and the data-generating process.

Figure lab: make the reversal visible

```
import matplotlib.pyplot as plt

def line_for(x, y, grid):
    b1 = slope_from_sums(x, y)
    b0 = y.mean() - b1 * x.mean()
    return b0 + b1 * grid

grid = np.linspace(0.5, 7.5, 100)

plt.figure(figsize=(8, 5))
plt.scatter(x_accessible, y_accessible, label="Accessible terrain", s=70)
plt.scatter(x_remote, y_remote, label="Remote terrain", s=70)
plt.plot(grid, line_for(x_accessible, y_accessible, grid), linewidth=2)
plt.plot(grid, line_for(x_remote, y_remote, grid), linewidth=2)
plt.plot(
    grid,
    line_for(x_all, y_all, grid),
    color="black",
    linestyle="--",
    label="Misleading combined line",
)
plt.xlabel("Cable length (km)")
plt.ylabel("Actual cost (million PKR)")
plt.title("Simpson's paradox in synthetic MHP projects")
plt.legend()
plt.tight_layout()
plt.show()
```

Figure 1.2 — Group-specific and aggregated regression lines. The dashed combined line answers a different—and substantively misleading—question.

1.7 A decision protocol before touching code

Use this sequence whenever someone asks for “a regression.”

Before analysis, write a one-sentence **estimand**—the exact quantity to be estimated. Examples:

- Prediction: “The expected final cost, in million PKR, of an approved project at the design stage.”
- Explanation: “The adjusted difference in final cost associated with one additional kilometre from an all-weather road, conditional on planned capacity and measured terrain difficulty.”

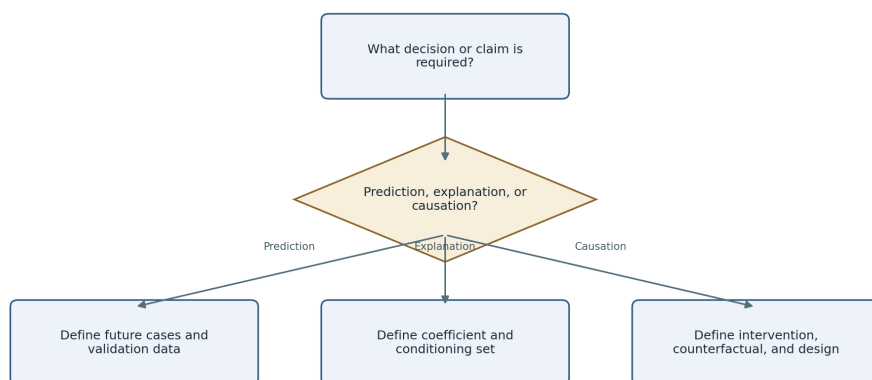


Figure 3: Concept diagram

- Causation: “The average change in final MHP cost caused by completing an access road before civil works, among currently approved remote projects.”

If the sentence is vague, the model will not rescue it.

1.8 Build: start the application with a question contract

Create `mhp_estimator.py`:

```

from dataclasses import dataclass
from typing import Literal

import numpy as np

AnalysisPurpose = Literal["prediction", "explanation", "causation"]

@dataclass(frozen=True)
class AnalysisContract:
    purpose: AnalysisPurpose
    target_name: str
    target_unit: str
    unit_of_observation: str
    intended_use: str

    def __post_init__(self):
        if self.purpose not in {"prediction", "explanation", "causation"}:
            raise ValueError("purpose must be prediction, explanation, or causation")
        for name, value in vars(self).items():
            if not str(value).strip():
                raise ValueError(f"{name} cannot be empty")

class MHPCostEstimator:
    """OLS estimator built progressively during Week 1."""

    def __init__(self, contract: AnalysisContract):
        self.contract = contract
  
```

```

self.parameters_ = None
self.feature_names_ = None
self.rank_ = None
self.is_fitted_ = False

def __repr__(self):
    status = "fitted" if self.is_fitted_ else "unfitted"
    return f"MHPCostEstimator(purpose={self.contract.purpose!r}, {status})"

if __name__ == "__main__":
    contract = AnalysisContract(
        purpose="prediction",
        target_name="actual_project_cost",
        target_unit="million PKR",
        unit_of_observation="one completed MHP project",
        intended_use="early budget review for approved projects",
    )
    print(MHPCostEstimator(contract))

```

@dataclass asks Python to generate routine methods such as `__init__` and `__repr__` from the listed fields. `frozen=True` prevents those fields from being reassigned after creation, which protects the analysis contract from accidental mid-analysis changes. `Literal[...]` is a type hint saying that readers and type-checking tools should expect exactly one of the three listed strings; the runtime check in `__post_init__` still performs the actual enforcement.

The class begins with a contract because a technically correct model can still be unfit for its intended use.

1.9 Break it deliberately

Change `purpose="prediction"` to `purpose="forecasting"`. The program should reject the unrecognised purpose. Then leave the purpose valid but make `intended_use=""`. It should reject the empty field.

The lesson is small but important: silent ambiguity is a data-quality problem.

1.10 Day 1 practice

1. A finance unit wants next year's total contingency allocation. Is its first task prediction, explanation, or causation?
2. A director asks whether contractor type "causes" delay because its regression coefficient is positive. What additional question must you ask?
3. Draw a DAG in which flood damage affects both the decision to redesign a headrace channel and final cost.
4. In the Simpson dataset, add 10 million PKR to every accessible project. Recalculate the three slopes. Which slopes change, and why?
5. Write an estimand for a real infrastructure decision you know.

Day 1 exit check

You are ready to continue if you can complete this sentence without hesitation:

"A regression coefficient is a causal effect only when _____."

A defensible completion is: "the design and assumptions make the comparison equivalent to the relevant intervention and counterfactual."

Day 2 — Regression Vocabulary and the Shape of Data

Today's central idea: Data have roles, shapes, units, and provenance. Most early modeling errors are representation errors before they are statistical errors.

2.1 The vocabulary of one project table

Suppose the rows of a table are completed MHP projects.

Term	Meaning	MHP example
Observation	One unit represented by one row	One completed MHP project
Feature	An input measured before the target is known	Planned capacity in kW
Target	The outcome to estimate	Actual project cost
Parameter	A value learned from data	Cost coefficient for road distance
Hyperparameter	A setting chosen outside the fitting calculation	Whether to include an intercept
Prediction	Model output for an observation	Estimated cost of 42 million PKR
Residual	Actual minus predicted target	$45 - 42 = 3$ million PKR
Loss	A rule for penalising errors during fitting	Squared residual
Metric	A summary used to evaluate performance	RMSE in million PKR
Estimator	The procedure that learns parameters	OLS
Fitted model	Estimator plus learned parameter values	Intercept and slopes after fitting

Parameter and **hyperparameter** should not be explained through a loose physical analogy. The reliable distinction is procedural: the fitting algorithm learns parameters from the training data; the analyst sets hyperparameters or design choices.

2.2 Scalars, vectors, matrices, and tensors

A **scalar** is one number. A **vector** is an ordered one-dimensional array. A **matrix** is a two-dimensional rectangular array. A **tensor** is the general term for an array with any number of axes.

The number of tensor axes is sometimes called its order or tensor rank. Do not confuse that usage with **matrix rank**, introduced on Day 5, which counts independent directions in a matrix.

Figure 2.1 — The dimensional ladder. Regression in this chapter mainly uses vectors and matrices; “tensor” is useful as the general family name.

The mathematical objects are:

$$X \in \mathbb{R}^{n \times p}, \quad y \in \mathbb{R}^n, \quad \beta \in \mathbb{R}^p.$$

Read them as:

**Figure 4:** Concept diagram

- X is a real-valued matrix with n rows and p feature columns;
- y is a real-valued target vector with n entries; and
- β is a parameter vector with one entry per feature.

Once an intercept column is added, X has $p + 1$ columns and β has $p + 1$ entries.

A concrete shape example

Three projects with two features—road distance and capacity—can be represented as:

$$X = \begin{bmatrix} 12 & 100 \\ 25 & 150 \\ 8 & 80 \end{bmatrix}, \quad y = \begin{bmatrix} 28 \\ 44 \\ 22 \end{bmatrix}.$$

Then X has shape 3×2 and y has length 3. Row 2 means that the second project is 25 km from an all-weather road, has planned capacity 150 kW, and actually cost 44 million PKR.

2.3 Shape is part of meaning

These arrays contain the same three numbers but do not have the same shape:

```
import numpy as np

a = np.array([28.0, 44.0, 22.0])           # shape (3,)
b = np.array([[28.0], [44.0], [22.0]])    # shape (3, 1)
c = np.array([28.0, 44.0, 22.0])          # shape (1, 3)

print(a.shape, b.shape, c.shape)
# (3,) (3, 1) (1, 3)
```

In NumPy, $(3,)$ is a one-dimensional vector. $(3, 1)$ is a two-dimensional column matrix. $(1, 3)$ is a two-dimensional row matrix. They can behave differently under multiplication and broadcasting.

Our application will accept a one-dimensional target vector y . $y.shape == (n,)$. It will convert a single feature from $(n,)$ to a two-dimensional matrix $(n, 1)$.

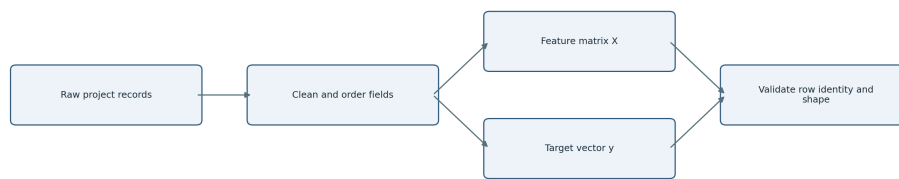
2.4 The design matrix and row alignment

The feature matrix used in a regression is called the **design matrix**. “Design” here refers to the arrangement of predictor columns; it does not mean the projects were experimentally assigned.

The i th row of X and the i th entry of y must refer to the same project.

Figure 2.2 — Data assembly. Matching row counts are necessary but not sufficient: row identities must also match.

If x is accidentally sorted by district while y remains sorted by project ID, the shapes still match. The code may run and the model may be nonsense. Preserve a project identifier during preparation, even if it is not used as a numerical feature.

**Figure 5:** Concept diagram

2.5 A data dictionary is part of the model

A usable dataset needs more than column names.

Field	Data type	Unit / categories	Timing	Missing-value rule
project_id	string	unique ID	approval	never missing
road_distance_km	float	km	design survey	verify from GIS if missing
planned_capacity_kw	float	kW	approved design	never impute casually
terrain_index	integer	1, 2, 3, 4, 5	pre-construction	document assessor
actual_cost_m_pkr	float	million PKR, stated price basis	completion	target; exclude incomplete projects

Timing prevents **target leakage**. A feature recorded after cost escalation—for example, “number of contract amendments”—might predict final cost extremely well. It is unavailable when the original budget is prepared and may partly be a consequence of the overrun. It should not be used in an early-stage prediction model without redefining the intended use.

2.6 Parsing one nested project record

```

import json
import numpy as np

raw_json = """
{
  "project_id": "MHP-CH-003",
  "design": {
    "road_distance_km": 18.5,
    "planned_capacity_kw": 120.0,
    "terrain_index": 4
  },
  "completion": {
    "actual_cost_m_pkr": 39.2
  }
}
"""

record = json.loads(raw_json)

X_one = np.array([[ # two brackets: one row, three columns
  record["design"]["road_distance_km"],

```

```

    record["design"]["planned_capacity_kw"],
    record["design"]["terrain_index"],
    ], dtype=float)

y_one = np.array([
    record["completion"]["actual_cost_m_pkr"]
], dtype=float)

print("X:", X_one, "shape:", X_one.shape)
print("y:", y_one, "shape:", y_one.shape)

assert X_one.shape == (1, 3)
assert y_one.shape == (1,)

```

The assert statements are executable claims. If the parser stops satisfying the intended mathematical shape, the program fails at the boundary instead of producing a later mystery.

2.7 Code proof: matrix-vector multiplication by loops

The linear prediction without an intercept is:

$$\hat{y}_i = \sum_{j=1}^p x_{ij} \beta_j.$$

For every project i , multiply each feature x_{ij} by its corresponding parameter β_j , then add the products.

```

X = np.array([
    [12.0, 100.0],
    [25.0, 150.0],
    [8.0, 80.0],
])
beta = np.array([0.6, 0.20])

# Proof using explicit loops
y_hat_loop = np.zeros(X.shape[0])
for i in range(X.shape[0]):
    total = 0.0
    for j in range(X.shape[1]):
        total += X[i, j] * beta[j]
    y_hat_loop[i] = total

# The same operation in matrix notation
y_hat_matrix = X @ beta

print(y_hat_loop)
print(y_hat_matrix)
assert np.allclose(y_hat_loop, y_hat_matrix)

```

The @ operator is compact, but it is not magic. The loops show exactly what it does.

2.8 Build: robust input validation

Add these methods inside `MHPCostEstimator`:

```

@staticmethod
def _validate_inputs(X, y=None):

```



```

"""Return finite float arrays with predictable regression shapes."""
X = np.asarray(X, dtype=float)

if X.ndim == 1:
    X = X.reshape(-1, 1)
if X.ndim != 2:
    raise ValueError(f"X must be 2D after conversion; got shape {X.shape}")
if X.shape[0] == 0 or X.shape[1] == 0:
    raise ValueError("X must contain at least one row and one feature")
if not np.isfinite(X).all():
    raise ValueError("X contains NaN or infinite values")

if y is None:
    return X

y = np.asarray(y, dtype=float)
if y.ndim == 2 and y.shape[1] == 1:
    y = y.reshape(-1)
if y.ndim != 1:
    raise ValueError(f"y must be 1D; got shape {y.shape}")
if X.shape[0] != y.shape[0]:
    raise ValueError(
        f"row mismatch: X has {X.shape[0]} rows but y has {y.shape[0]}"
    )
if not np.isfinite(y).all():
    raise ValueError("y contains NaN or infinite values")

return X, y

```

Why each check exists

- `dtype=float` prevents accidental string arithmetic.
- A 1D `x` is interpreted as one feature observed for many projects.
- Regression requires a 2D design matrix even when $p = 1$.
- Empty arrays have mathematically valid-looking shapes but no estimable relationship.
- NaN and infinity propagate through calculations and can silently destroy results.
- A one-column target matrix is flattened to the application's chosen convention.
- Matching row counts are enforced, though project-ID alignment must be checked earlier.

2.9 Break it deliberately

Run each case and predict the error message before reading it.

```

contract = AnalysisContract(
    purpose="prediction",
    target_name="actual_cost_m_pkr",
    target_unit="million PKR",
    unit_of_observation="completed MHP project",
    intended_use="design-stage budget review",
)
model = MHPCostEstimator(contract)

bad_cases = [
    (np.ones((10, 3)), np.ones(8)),          # row mismatch
    (np.array([[1.0, np.nan]]), [2.0]),      # missing feature
    (np.empty((0, 2)), np.empty(0)),          # no observations
    (np.ones((2, 2, 2)), np.ones(2)),        # three-dimensional X

```

```

]

for X_bad, y_bad in bad_cases:
    try:
        model._validate_inputs(X_bad, y_bad)
    except ValueError as error:
        print(type(error).__name__, "->", error)

```

Then construct the more dangerous failure: create X and y with the same number of rows but place y in reverse project order. Validation cannot detect the mistake unless project identifiers are checked during data assembly.

2.10 Day 2 practice

1. State the shape of a dataset containing 70 projects and 5 features.
2. After adding an intercept, what are the shapes of X and β ?
3. Why is `project_id` important even when it is not a numerical feature?
4. Give one example of target leakage in an MHP cost model.
5. Extend `_validate_inputs` to reject a constant target vector, but explain why a constant target is not mathematically invalid in every context.

Day 2 exit check

Given `X.shape == (40, 6)` and `beta.shape == (5,)`, you should immediately say: “Multiplication is impossible because the inner dimensions, 6 and 5, do not match.”

Day 3 — Linear Algebra as a System of Predictions

Today’s central idea: A linear model forms each prediction by adding weighted features. Matrix notation performs that same operation for every project at once.

3.0 Matplotlib in five lines

Matplotlib turns arrays into figures. `plt.figure()` starts a canvas, `scatter` draws points, `plot` joins coordinates with a line, `legend` labels the layers, and `show` displays the finished result.

```

import matplotlib.pyplot as plt

x_demo = [0, 1, 2]
y_demo = [1, 3, 5]
plt.figure(figsize=(5, 3))
plt.scatter(x_demo, y_demo, label="observations")
plt.plot(x_demo, y_demo, color="darkorange", label="joined points")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()

```

Every later figure repeats this pattern with different drawing commands. Run this tiny example first; if no window or notebook figure appears, fix the plotting setup before continuing.

3.0A Visual mathematics bridge: from a map to a graph

Before learning regression, we need a small vocabulary for pictures made from numbers.

A **map** and a **graph** both use position to communicate information, but their positions mean different things.

Picture	Horizontal position means	Vertical position means
Geographic map	Usually east-west location	Usually north-south location
Cost graph	A chosen feature, such as cable length	A chosen outcome, such as project cost

This distinction matters. A point on a cost graph is not the physical location of a project. It is a compact numerical description of that project.

Suppose Project A uses 2 km of distribution cable and costs 20 million PKR. We represent it by the ordered pair:

$$(x, y) = (2, 20).$$

The order is fixed: first the horizontal coordinate, then the vertical coordinate. Here:

- move 2 units along the horizontal cable-length **axis**;
- then move 20 units along the vertical cost axis; and
- place the point where those two positions meet.

An **axis** is simply a numbered reference line. The horizontal axis is conventionally called the x -axis and the vertical axis the y -axis. Their crossing point, $(0, 0)$, is the **origin**.

Do not attach permanent meanings to the letters. In this example x means cable length and y means cost. In another graph, x could mean distance from a road and y could mean construction time. Always read the axis labels and units.

Points, a line, and a relationship

Consider four exact points:

Cable length x (km)	Cost y (million PKR)	Coordinate
1	12	$(1, 12)$
2	16	$(2, 16)$
3	20	$(3, 20)$
4	24	$(4, 24)$

Each additional kilometre goes with an additional 4 million PKR. These points lie on the line:

$$y = 8 + 4x.$$

A **straight line** has a constant rate of change: equal horizontal steps produce equal vertical steps. If x increases by 1 km anywhere on this line, y increases by 4 million PKR.

3.0B Slope: rise divided by run

The number describing a line's steepness and direction is its **slope**.

Choose any two distinct points on a straight line:

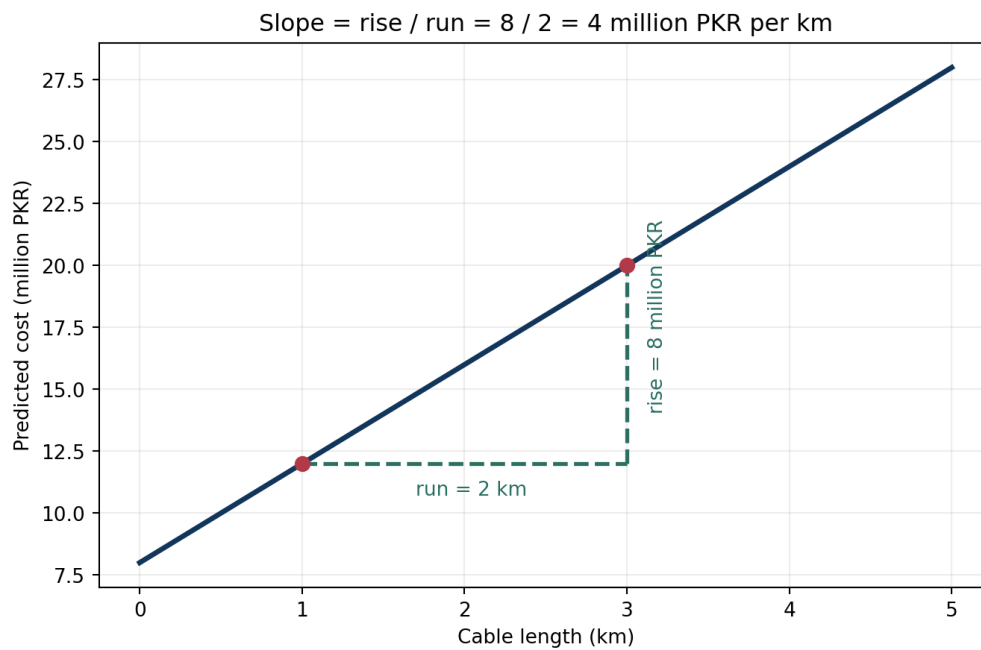


Figure 6: Slope shown as rise divided by run.

$$(x_1, y_1) \text{ and } (x_2, y_2).$$

The horizontal change is called the **run**:

$$\text{run} = x_2 - x_1.$$

The vertical change is called the **rise**:

$$\text{rise} = y_2 - y_1.$$

The slope is:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{y_2 - y_1}{x_2 - x_1}$$

For the points (1, 12) and (3, 20):

$$\text{run} = 3 - 1 = 2 \text{ km},$$

$$\text{rise} = 20 - 12 = 8 \text{ million PKR},$$

so:

$$\text{slope} = \frac{8 \text{ million PKR}}{2 \text{ km}} = 4 \text{ million PKR per km}.$$

The units are part of the answer. A bare statement that “the slope is 4” is incomplete.

How to read the sign of a slope

Slope	What the line does as x increases	Example reading
Positive	Rises from left to right	Greater road distance is associated with greater predicted cost
Zero	Remains horizontal	Predicted cost does not change with the feature
Negative	Falls from left to right	Greater road access is associated with lower predicted transport time
Undefined	Is vertical; run is zero	Several y -values are placed at one x -value, so rise/0 cannot be computed

“Steeper” means a larger absolute change in y per unit of x . A slope of -6 falls more steeply than a slope of -2 , although -6 is numerically smaller.

Why the same line gives the same slope everywhere

On a straight line, any two points give the same rise-to-run ratio. If a 1 km movement raises predicted cost by 4 million PKR, then a 3 km movement raises it by 12 million PKR:

$$\frac{4}{1} = \frac{12}{3} = 4.$$

That constant ratio is what makes the relationship linear.

Figure lab: draw the slope triangle

```
import matplotlib.pyplot as plt
import numpy as np

x_line = np.linspace(0, 5, 100)
y_line = 8 + 4 * x_line

x1, y1 = 1, 12
x2, y2 = 3, 20

plt.figure(figsize=(7, 5))
plt.plot(x_line, y_line, color="navy", label=r"$y=8+4x$")
plt.scatter([x1, x2], [y1, y2], color="firebrick", zorder=3)
plt.plot([x1, x2], [y1, y1], "--", color="darkgreen")
plt.plot([x2, x2], [y1, y2], "--", color="darkgreen")
plt.text(1.8, 11.1, "run = 2 km")
plt.text(3.1, 15.5, "rise = 8 million PKR", rotation=90)
plt.xlabel("Cable length (km)")
plt.ylabel("Predicted cost (million PKR)")
plt.title("Slope is rise divided by run")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

3.0C Intercept: where the line meets an axis

In:

$$y = 8 + 4x,$$

the line meets the vertical axis when $x = 0$:

$$y = 8 + 4(0) = 8.$$

The number 8 is the **vertical intercept**, usually shortened to **intercept**. In the regression notation:

$$\hat{y} = \beta_0 + \beta_1 x,$$

- β_0 is the intercept; and
- β_1 is the slope.

The hat over y matters. The line describes the model's **predicted** value $\hat{y}$, not a promise that every actual observation y lies on the line.

A geographical caution about an intercept

If every observed MHP project lies between 1 and 12 km from a road, the point “road distance = 0” is outside the observed range. The intercept is still needed to position the fitted line, but it may not be a credible estimate of the cost of a road-side project. This is the first example of **extrapolation**: asking the model about a place in numerical space where it has little or no supporting data.

3.0D Curves, secants, and tangents

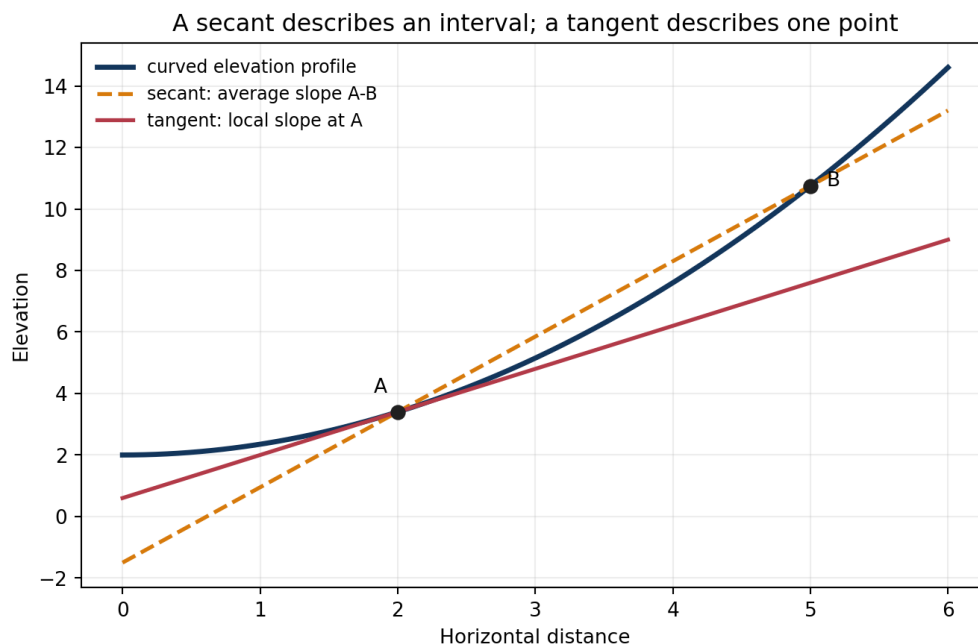


Figure 7: A secant measures average change across an interval; a tangent measures local change at one point.

A line has one constant slope. A **curve** can be steep in one place, flat in another, and steep again later.

Mountain travel provides an intuitive example. Let:

- horizontal position represent distance travelled along a valley; and
- vertical position represent elevation.

The elevation profile bends because the route climbs gently in some stretches and sharply in others.

Secant: average slope between two points

Choose two points, A and B, on the curve and draw a straight line through them. That line is a **secant**. Its slope is:

$$\text{average slope from A to B} = \frac{\text{change in elevation}}{\text{change in horizontal distance}}.$$

It summarises the whole interval. It does not tell us the exact steepness at a particular bend between A and B.

Tangent: local direction at one point

A **tangent** is a straight line that matches the direction of a smooth curve at one chosen point. Imagine placing a short ruler against a smooth road profile so that, at the chosen point, the ruler points in the same direction as the road.

The tangent need not remain close to the curve far away. Its purpose is local: it tells us the curve's direction and steepness at that point.

We can approach the tangent without advanced mathematics:

1. Pick a point A on the curve.
2. Pick a second point B nearby.
3. Calculate the secant slope from A to B.
4. Move B closer to A and recalculate.
5. The value approached by those secant slopes is the **tangent slope** at A.

That limiting tangent slope is what calculus calls a **derivative**.

Figure lab: secant becoming tangent

```
import matplotlib.pyplot as plt
import numpy as np

def elevation(distance):
    return 0.35 * distance ** 2 + 2

x_curve = np.linspace(0, 6, 300)
x_a = 2.0
x_b = 5.0
y_a = elevation(x_a)
y_b = elevation(x_b)
secant_slope = (y_b - y_a) / (x_b - x_a)
tangent_slope = 0.7 * x_a

secant = y_a + secant_slope * (x_curve - x_a)
tangent = y_a + tangent_slope * (x_curve - x_a)

plt.figure(figsize=(7, 5))
plt.plot(x_curve, elevation(x_curve), color="navy", label="elevation curve")
plt.plot(x_curve, secant, "--", color="darkorange", label="secant A-B")
plt.plot(x_curve, tangent, color="firebrick", label="tangent at A")
```

```
plt.scatter([x_a, x_b], [y_a, y_b], color="black", zorder=3)
plt.text(x_a - 0.2, y_a + 0.5, "A")
plt.text(x_b + 0.1, y_b, "B")
plt.xlabel("Horizontal distance")
plt.ylabel("Elevation")
plt.title("A secant measures an interval; a tangent measures one point")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

Important distinction: The regression **slope** β_1 is the constant slope of a fitted straight line in the data graph. The derivative $dS/d\beta$ used on Day 5 is the local slope of a different curve: the curve that relates a proposed coefficient β to the model's total squared error S . Both are called slopes, but they answer different questions.

3.0E Geography is more than a point on a map

In this chapter, “geographical explanation” means more than adding latitude and longitude. Geography shapes how MHP projects are produced and observed:

- a kilometre of cable across an accessible plain is not operationally equivalent to a kilometre across a gorge;
- road distance may understate access difficulty when a bridge is absent or a pass closes in winter;
- projects in one valley may share contractors, prices, weather shocks, and administrative systems;
- remote projects may be selected only when donor funding is unusually high, so observed remote projects are not a random sample of all possible remote projects; and
- a model learned in Swat may not transfer unchanged to upper Chitral, even if both datasets contain columns with the same names.

OLS sees only the numbers placed in X and y . It does not see a mountain, a clan boundary, a washed-out bridge, a winter closure, or a weak procurement market unless the data representation captures those conditions. A geographically informed analyst therefore asks two questions:

1. **Geometric question:** Which fitted line or prediction vector is closest to the observed data?
2. **Geographical question:** Why did these projects, features, costs, and omissions take this particular form in these particular places?

The first is answered by OLS mathematics. The second requires field knowledge, institutional understanding, and a defensible data-generating story.

3.1 From a verbal rule to a linear equation

Suppose a deliberately simple budget rule uses only cable length:

Begin with a fixed base cost of 8 million PKR, then add 4 million PKR for each kilometre of cable.

For project i :

$$\hat{y}_i = 8 + 4x_i.$$

The general one-feature linear model is:

$$\hat{y}_i = \beta_0 + \beta_1 x_i.$$

- β_0 is the **intercept**: the model's predicted target when $x_i = 0$.
- β_1 is the **slope**: the predicted change in y associated with a one-unit increase in x .

If y is million PKR and x is km, the slope's units are million PKR per km.

The intercept may be necessary for good predictions without having a useful physical interpretation. "A project with zero cable" may lie outside the scope of the data. An intercept is an algebraic baseline, not automatically the price of a literal zero-feature project.

3.2 Multiple features are still a weighted sum

With road distance, capacity, and terrain difficulty:

$$\hat{y}_i = \beta_0 + \beta_1 \text{road_distance}_i + \beta_2 \text{capacity}_i + \beta_3 \text{terrain}_i.$$

For one project, this is a dot product between a row of feature values and a vector of parameters. For every project, it becomes matrix-vector multiplication:

$$\hat{y} = X\beta.$$

The apparent simplicity of the matrix equation is earned by carefully constructing X .

3.3 Why a column of ones creates an intercept

For three projects and one measured feature, start with:

$$X_{\text{raw}} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}.$$

Prepend a column of ones:

$$X = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}.$$

Now multiply row by column:

$$X\beta = \begin{bmatrix} 1\beta_0 + x_1\beta_1 \\ 1\beta_0 + x_2\beta_1 \\ 1\beta_0 + x_3\beta_1 \end{bmatrix}.$$

The same β_0 is added to every project because every first-column entry is 1.

Without the column of ones, the one-feature model is $\hat{y} = \beta_1 x$. At $x = 0$, it must pass through the origin. With the ones column, the model is **affine**: a linear transformation plus a translation. In everyday statistics it is still called a linear regression because it is linear in the parameters.

3.4 Code proof: expand $X @ \beta$ by hand

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
```

```

X = np.column_stack([np.ones(x.size), x])
beta = np.array([8.0, 4.0])

y_hat_matrix = X @ beta
y_hat_manual = np.array([
    1.0 * 8.0 + 1.0 * 4.0,
    1.0 * 8.0 + 2.0 * 4.0,
    1.0 * 8.0 + 3.0 * 4.0,
])

print(X)
print(y_hat_matrix)
assert np.array_equal(y_hat_matrix, y_hat_manual)
assert np.array_equal(y_hat_matrix, np.array([12.0, 16.0, 20.0]))

```

Every multiplication in the displayed equation appears explicitly in the code.

3.5 Shape algebra before numerical algebra

If:

$$X \in \mathbb{R}^{n \times (p+1)} \quad \text{and} \quad \beta \in \mathbb{R}^{p+1},$$

then:

$$X\beta \in \mathbb{R}^n.$$

The inner dimensions must agree:

$$(n \times (p+1)) (p+1 \times 1) = n \times 1.$$

This is not a software convention. It follows from the dot product: each row of X needs exactly one parameter for each column.

Symbol-to-Python map

Mathematics	Meaning	NumPy
$X\beta$	Matrix-vector product	<code>X @ beta</code>
X^T	Transpose	<code>X.T</code>
$x^T y$	Dot product	<code>x @ y</code> or <code>np.dot(x, y)</code>
I	Identity matrix	<code>np.eye(p)</code>
$\mathbf{1}$	Vector of ones	<code>np.ones(n)</code>
$\ v\ _2$	Euclidean length	<code>np.linalg.norm(v)</code>

3.6 Matrices as transformations—and the limit of the analogy

A matrix can transform geometric coordinates by stretching, shrinking, rotating, reflecting, or shearing vectors. That view is useful for understanding linear algebra. In regression, however, the most immediate interpretation of $X\beta$ is slightly different: the columns of X are building blocks, and β chooses how much of each column to combine.

If X has columns $x_0, x_1, \dots, x_p$, then:

$$X\beta = \beta_0 x_0 + \beta_1 x_1 + \dots + \beta_p x_p.$$

Because x_0 is the column of ones, $\beta_0 x_0$ adds the same amount to every prediction.

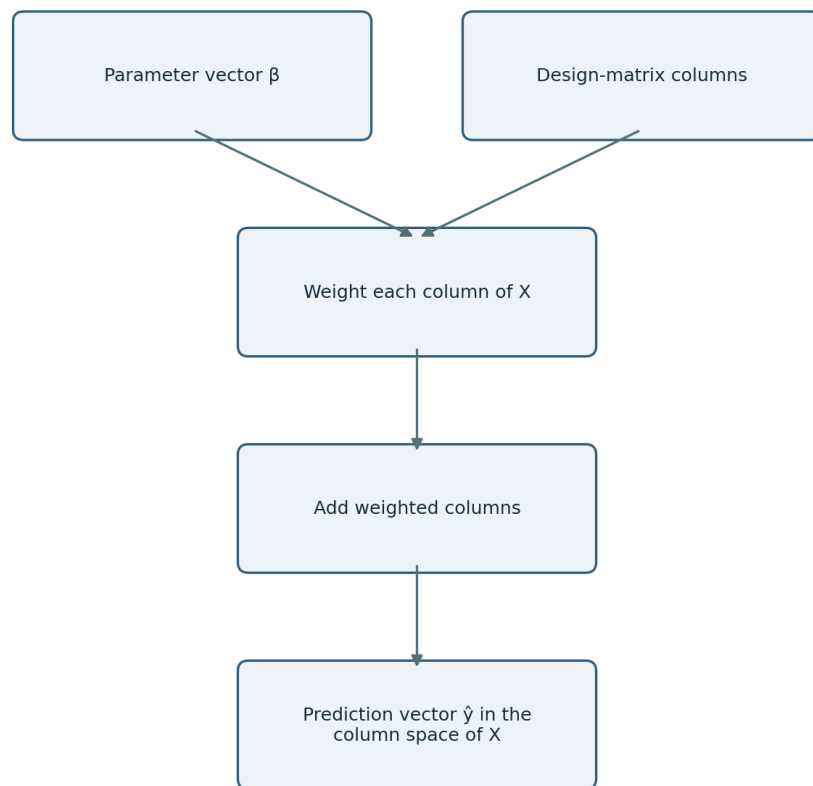


Figure 8: Concept diagram

Figure 3.1 — Column-combination view of regression. The prediction vector is not an arbitrary vector; it must be built from the columns available in X .

3.7 Figure lab: with and without an intercept

Use four synthetic projects:

```
import matplotlib.pyplot as plt
import numpy as np

x_cable_km = np.array([1.0, 2.0, 3.0, 4.0])
y_cost_m_pkr = np.array([12.0, 19.0, 29.0, 38.0])

# OLS through the origin: minimise ||y - xb||^2
b_origin = (x_cable_km @ y_cost_m_pkr) / (x_cable_km @ x_cable_km)

# OLS with an intercept: solve the normal equations
X = np.column_stack([np.ones(x_cable_km.size), x_cable_km])
beta = np.linalg.solve(X.T @ X, X.T @ y_cost_m_pkr)

grid = np.linspace(0, 5, 100)

plt.figure(figsize=(8, 5))
```

```
plt.scatter(x_cable_km, y_cost_m_pkr, color="navy", s=70, label="Projects")
plt.plot(grid, b_origin * grid, "--", color="firebrick", label="Forced through origin")
plt.plot(grid, beta[0] + beta[1] * grid, color="darkgreen", label="With intercept")
plt.axhline(0, color="grey", linewidth=0.6)
plt.axvline(0, color="grey", linewidth=0.6)
plt.xlabel("Cable length (km)")
plt.ylabel("Actual cost (million PKR)")
plt.title("The intercept translates the fitted line")
plt.legend()
plt.tight_layout()
plt.show()

print("Through-origin slope:", b_origin)
print("Intercept and slope:", beta)
```

Figure 3.2 — The effect of the intercept. The through-origin model is required to predict zero cost at zero cable length. The intercept model is allowed to fit the observed baseline.

Notice that the code uses `np.linalg.solve(A, b)` for $A\beta = b$, not `np.linalg.inv(A) @ b`. The inverse formula is useful for derivation, but directly solving the linear system is clearer and usually more numerically stable.

3.8 Scaling and units

Suppose cable is changed from kilometres to metres:

$$x_{\text{metres}} = 1000x_{\text{km}}.$$

To preserve the same prediction:

$$\beta_{\text{metres}} = \frac{\beta_{\text{km}}}{1000}.$$

Then:

$$\beta_{\text{metres}}x_{\text{metres}} = \frac{\beta_{\text{km}}}{1000}(1000x_{\text{km}}) = \beta_{\text{km}}x_{\text{km}}.$$

Code proof: changing units changes the coefficient, not the prediction

```
x_km = np.array([1.0, 2.0, 3.0])
x_m = 1000.0 * x_km

beta_km = 4.0          # million PKR per km
beta_m = beta_km / 1000.0

assert np.allclose(beta_km * x_km, beta_m * x_m)
```

This is why comparing coefficient magnitudes across differently scaled features is misleading. A smaller numerical coefficient need not represent a less important variable.

3.9 Build: intercept construction and prediction

Add these methods inside `MHPCostEstimator`:

```
@staticmethod
def _add_intercept_column(X):
```

```

"""Prepend one constant column; X must already be validated and 2D."""
ones = np.ones((X.shape[0], 1), dtype=float)
return np.hstack([ones, X])

def predict(self, X):
    """Return predictions for new feature rows."""
    if not self.is_fitted_:
        raise RuntimeError("fit the estimator before calling predict")

    X = np.asarray(X, dtype=float)
    if X.ndim == 1:
        X = X.reshape(1, -1) # one new project with p features
    X = self._validate_inputs(X)
    if X.shape[1] != len(self.feature_names_):
        raise ValueError(
            f"expected {len(self.feature_names_)} features, got {X.shape[1]}"
        )

    X_design = self._add_intercept_column(X)
    return X_design @ self.parameters_

```

predict cannot yet succeed because fit has not been built. This is intentional. We have defined the behaviour of a fitted application before learning how to estimate the parameters.

The context resolves a genuine shape ambiguity: during fitting, a one-dimensional x means one feature observed across many projects; during prediction, a one-dimensional x means one new project containing several features. Passing an explicit two-dimensional array remains safest.

3.10 Break it deliberately: broadcasting is not matrix multiplication

```

X = np.array([
    [1.0, 12.0, 100.0],
    [1.0, 25.0, 150.0],
])
beta = np.array([5.0, 0.6, 0.2])

correct = X @ beta
print("correct shape:", correct.shape, "values:", correct)

elementwise = X * beta
print("elementwise shape:", elementwise.shape)
print(elementwise)

```

$X * \text{beta}$ broadcasts the parameter vector across rows and multiplies element by element. It returns a matrix of products, not a prediction vector. The expression runs without an error, which makes it more dangerous than a crash.

Also try multiplying the same X by $\text{np.array}([5.0, 0.6])$. Matrix multiplication should fail because the number of parameters does not equal the number of columns.

3.11 Day 3 practice

1. Expand one row of $X\beta$ for an intercept plus three features.
2. Explain why a column of twos would not estimate the conventional intercept. What would its coefficient represent?

3. If cost changes from million PKR to PKR, how do the coefficients change?
4. Write a `matvec_with_loops(X, beta)` function that rejects incompatible dimensions.
5. Plot a dataset for which the intercept has a strong effect on fit and another for which it has little effect.

Day 3 exit check

You should be able to explain both statements:

- “ $X\beta$ is one dot product per project.”
- “ $X\beta$ is a weighted combination of the columns of X .”

They are two views of the same operation.

Day 4 — Residuals, Loss, and the Geometry of OLS

Today’s central idea: OLS chooses the attainable prediction vector closest to the observed target vector in squared Euclidean distance.

4.1 A prediction becomes testable through its residual

For project i :

$$e_i = y_i - \hat{y}_i.$$

- $e_i > 0$: actual cost exceeded predicted cost; the model underpredicted.
- $e_i < 0$: actual cost was below predicted cost; the model overpredicted.
- $e_i = 0$: prediction matched the observed target.

The sign convention matters. This chapter uses **actual minus predicted**.

For all projects:

$$e = y - \hat{y} = y - X\beta.$$

Code proof: the vector equation and the elementwise equation agree

```
import numpy as np

y = np.array([12.0, 19.0, 29.0])
y_hat = np.array([10.5, 20.0, 27.5])

e_vector = y - y_hat
e_manual = np.array([
    12.0 - 10.5,
    19.0 - 20.0,
    29.0 - 27.5,
])

assert np.allclose(e_vector, e_manual)
print(e_vector) # [ 1.5 -1.   1.5]
```

Residuals are observed after fitting. They are not the same as the unobservable population errors in a statistical model, though introductory discussions often use the words loosely.

4.2 Why OLS squares residuals

The **sum of squared residuals** (SSR), also called the residual sum of squares (RSS) or sum of squared errors (SSE), is:

$$SSR(\beta) = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - x_i^T \beta)^2.$$

In vector notation:

$$SSR(\beta) = e^T e = (y - X\beta)^T (y - X\beta).$$

Squaring has four important consequences:

1. Positive and negative residuals cannot cancel.
2. Large residuals receive disproportionate weight.
3. The objective is smooth and differentiable everywhere.
4. The objective has a geometric interpretation as squared Euclidean distance.

The second consequence is not automatically desirable. A data-entry error of 390 million instead of 39 million can dominate an OLS fit. High stakes do not, by themselves, prove that squared loss is the right institutional loss function. The analyst must inspect outliers and ask whether the objective reflects the decision.

Code proof: $e^T e$ equals the sum of elementwise squares

```
e = np.array([1.5, -1.0, 1.5])

ssr_loop = 0.0
for value in e:
    ssr_loop += value ** 2

ssr_sum = np.sum(e ** 2)
ssr_dot = e.T @ e

print(ssr_loop, ssr_sum, ssr_dot)
assert np.isclose(ssr_loop, ssr_sum)
assert np.isclose(ssr_sum, ssr_dot)
```

4.3 SSR, MSE, RMSE, and MAE answer different reporting needs

$$MSE = \frac{1}{n} \sum_{i=1}^n e_i^2,$$

$$RMSE = \sqrt{MSE},$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |e_i|.$$

Metric	Units	Sensitivity to large residuals	Typical use
SSR	squared target units	high	Fitting and comparing models on the same observations
MSE	squared target units	high	Average squared loss
RMSE	target units	high	Error summary interpretable in cost units
MAE	target units	lower	Typical absolute miss, more robust than RMSE

With targets in million PKR, RMSE and MAE are also in million PKR. MSE is in squared million PKR, which is harder to communicate.

Code proof: compute every metric from definitions

```
def regression_metrics(y, y_hat):
    y = np.asarray(y, dtype=float)
    y_hat = np.asarray(y_hat, dtype=float)
    if y.shape != y_hat.shape:
        raise ValueError("y and y_hat must have the same shape")

    residuals = y - y_hat
    ssr = np.sum(residuals ** 2)
    mse = ssr / residuals.size
    rmse = np.sqrt(mse)
    mae = np.sum(np.abs(residuals)) / residuals.size
    return {"ssr": ssr, "mse": mse, "rmse": rmse, "mae": mae}

metrics = regression_metrics(
    y=np.array([12.0, 19.0, 29.0]),
    y_hat=np.array([10.5, 20.0, 27.5]),
)
print(metrics)
```

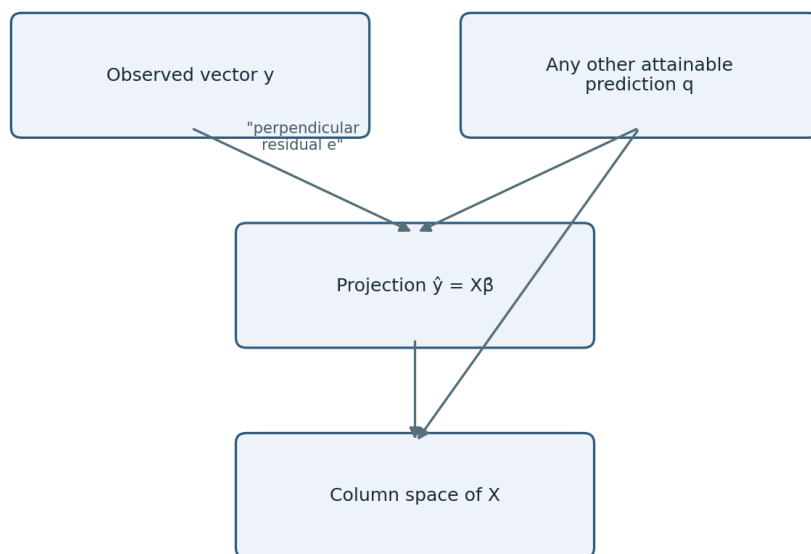
For training data, some texts divide SSR by $n - p - 1$ to estimate error variance. That is a different quantity from predictive MSE. Always name the denominator.

4.4 OLS as a constrained search in observation space

The target vector y contains one coordinate for each project, so it lives in $\mathbb{R}^n$. Every possible $X\beta$ is a weighted combination of the columns of X . The set of all such combinations is the **column space** of X .

OLS cannot choose any point in $\mathbb{R}^n$. It must choose a point inside that column space. It selects the point closest to y .

Figure 4.1 — Projection view of OLS. The diagram is conceptual: $\hat{y}$ and q lie in the column space, while the residual from $\hat{y}$ to y is perpendicular to it.

**Figure 9:** Concept diagram

4.5 The Pythagorean proof of minimum distance

Let:

- $\hat{y}$ be the perpendicular projection of y onto the column space of X ;
- q be any other prediction in the column space;
- $e = y - \hat{y}$ be the OLS residual; and
- $d = \hat{y} - q$ be the displacement within the column space.

Then:

$$y - q = (y - \hat{y}) + (\hat{y} - q) = e + d.$$

Because e is perpendicular to the entire column space and d lies inside it:

$$e^T d = 0.$$

Now expand the squared distance:

$$\begin{aligned}
 \|y - q\|_2^2 &= (e + d)^T (e + d) \\
 &= e^T e + e^T d + d^T e + d^T d \\
 &= \|e\|_2^2 + \|d\|_2^2 \\
 &\geq \|e\|_2^2.
 \end{aligned}$$

The last step holds because a squared length cannot be negative. Therefore no other attainable prediction q can be closer to y than $\hat{y}$.

4.6 Why orthogonality produces the normal equations

Every column of X lies in the column space. If the residual is perpendicular to the whole column space, it is perpendicular to every column:

$$X^T e = 0.$$

Substitute $e = y - X\hat{\beta}$:

$$X^T(y - X\hat{\beta}) = 0.$$

Distribute X^T :

$$X^T y - X^T X \hat{\beta} = 0.$$

Rearrange:

$$X^T X \hat{\beta} = X^T y.$$

These are the **normal equations**. “Normal” refers to perpendicularity, not to the normal probability distribution.

If an intercept column is present, one of the orthogonality conditions is:

$$\mathbf{1}^T e = \sum_{i=1}^n e_i = 0.$$

Therefore OLS residuals from the training data sum to zero when the model includes an intercept, up to floating-point error.

4.7 Code proof: orthogonality and Pythagoras

```
import numpy as np

x = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([12.0, 19.0, 29.0, 38.0])
X = np.column_stack([np.ones(x.size), x])

beta_hat = np.linalg.solve(X.T @ X, X.T @ y)
y_hat = X @ beta_hat
e = y - y_hat

# Orthogonality: one near-zero dot product per design column
print("X.T @ e =", X.T @ e)
assert np.allclose(X.T @ e, np.zeros(X.shape[1]), atol=1e-10)

# Intercept implication: residuals sum to zero
assert np.isclose(e.sum(), 0.0, atol=1e-10)

# Choose another attainable prediction q = X @ beta_other
beta_other = beta_hat + np.array([2.0, -1.0])
q = X @ beta_other
d = y_hat - q

left = np.linalg.norm(y - q) ** 2
right = np.linalg.norm(e) ** 2 + np.linalg.norm(d) ** 2
```

```
print("Pythagorean sides:", left, right)
assert np.isclose(e @ d, 0.0, atol=1e-10)
assert np.isclose(left, right, atol=1e-10)
assert left >= np.linalg.norm(e) ** 2
```

This block verifies three equations: the normal-equation orthogonality condition, the zero-sum residual property, and the Pythagorean decomposition.

4.8 Figure lab: draw the projection in three dimensions

Optional visual extension. This 3D plot is useful intuition but uses advanced plotting APIs that are not required for the algebra, code, or exit check. Skip it on a first pass if *quiver*, *plot_surface*, or 3D axes distract from the projection idea.

Three observations allow the target and prediction vectors to be drawn in $\mathbb{R}^3$.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([-1.0, 0.0, 1.0])
X = np.column_stack([np.ones(3), x])
y = np.array([1.0, 3.0, 2.0])

beta_hat = np.linalg.solve(X.T @ X, X.T @ y)
y_hat = X @ beta_hat
e = y - y_hat

fig = plt.figure(figsize=(8, 7))
ax = fig.add_subplot(111, projection="3d")
origin = np.zeros(3)

ax.quiver(*origin, *y, color="navy", label="Observed y", arrow_length_ratio=0.08)
ax.quiver(*origin, *y_hat, color="darkgreen", label="Projection  $\hat{y}$ ", arrow_length_ratio=0.08)
ax.quiver(*y_hat, *e, color="firebrick", label="Residual e", arrow_length_ratio=0.15)

# Draw a patch of the column space span{ones, x}
a = np.linspace(-0.5, 2.5, 10)
b = np.linspace(-1.0, 1.0, 10)
A, B = np.meshgrid(a, b)
plane = A[..., None] * X[:, 0] + B[..., None] * X[:, 1]
ax.plot_surface(plane[..., 0], plane[..., 1], plane[..., 2], alpha=0.2)

ax.set_xlabel("Project 1 coordinate")
ax.set_ylabel("Project 2 coordinate")
ax.set_zlabel("Project 3 coordinate")
ax.set_title("OLS projects y onto the column space of X")
ax.legend()
plt.tight_layout()
plt.show()
```

Here `[..., None]` keeps every existing axis and adds one new length-one axis at the end. That makes NumPy broadcast each grid coefficient across the three coordinates of the vector it multiplies.

Figure 4.2 — OLS geometry in observation space. The residual begins at $\hat{y}$ and ends at y ; it is perpendicular to the surface of attainable predictions.

4.9 Build: metrics as methods

Add these methods to `MHPCostEstimator`:

```
@staticmethod
def _metrics_from_predictions(y, y_hat):
    y = np.asarray(y, dtype=float).reshape(-1)
    y_hat = np.asarray(y_hat, dtype=float).reshape(-1)
    if y.shape != y_hat.shape:
        raise ValueError("y and y_hat must have identical shapes")

    residuals = y - y_hat
    ssr = float(residuals @ residuals)
    mse = ssr / residuals.size
    return {
        "ssr": ssr,
        "mse": mse,
        "rmse": float(np.sqrt(mse)),
        "mae": float(np.mean(np.abs(residuals))),
    }

def evaluate(self, X, y):
    """Evaluate a fitted model on supplied observations."""
    X, y = self._validate_inputs(X, y)
    y_hat = self.predict(X)
    return self._metrics_from_predictions(y, y_hat)
```

The method does not call these “test metrics” because the caller may supply training, validation, or test data. The application should not pretend to know the provenance of an array.

4.10 Break it deliberately: one outlier

```
y = np.array([20.0, 22.0, 24.0, 26.0])
y_hat = np.array([21.0, 21.0, 25.0, 25.0])

ordinary = regression_metrics(y, y_hat)

y_with_error = y.copy()
y_with_error[-1] = 260.0 # possible extra zero in data entry
contaminated = regression_metrics(y_with_error, y_hat)

print("Ordinary: ", ordinary)
print("Contaminated:", contaminated)
```

Compare the change in RMSE and MAE. Do not conclude merely that MAE is “better.” First determine whether 260 is a genuine extreme project, a different type of project, or an error. Robustness cannot replace data investigation.

4.11 Day 4 practice

1. Calculate the residuals and SSR for $y = [3, 5]$ and $\hat{y} = [4, 3]$.
2. Prove in code that RMSE has the same units as the target under a change from million PKR to PKR.
3. Why does including an intercept make training residuals sum to zero?
4. Does a zero sum of residuals imply a good model? Construct a counterexample.
5. Modify Figure 4.2 by choosing a new y . Verify that $X.T @ e$ remains near zero.

Day 4 exit check

You should be able to explain OLS without calculus:

“Among all prediction vectors that can be constructed from the columns of X , OLS chooses the one with the shortest squared distance to y .”

Day 5 — Deriving, Fitting, and Stress-Testing OLS

Today’s central idea: OLS is the parameter value that makes the squared-error surface as small as possible. The normal equation is a consequence, not a formula to memorise.

Day 4 reached the normal equations through geometry:

$$X^T X \hat{\beta} = X^T y.$$

Today we reach the same point through calculus, first with one feature and then with matrices. The two derivations are complementary:

- geometry explains **what** OLS is doing; and
- calculus explains **how** the optimum is located.

5.0 Calculus bridge: change, tangent, derivative, and minimum

The word **calculus** can make a simple idea sound remote. For this chapter, calculus is a precise language for answering one question:

If I change a proposed coefficient a little, in which direction and how quickly will the total squared error change?

No prior calculus is assumed. We begin with the picture.

5.0.1 A function is an input-output rule

A **function** accepts an input and produces exactly one output. For example:

$$f(x) = 8 + 4x$$

turns a cable length into a predicted cost. If $x = 3$:

$$f(3) = 8 + 4(3) = 20.$$

OLS uses another function:

$$S(\beta) = \text{total squared error produced by the proposed coefficient } \beta.$$

Here the input is not a cable length. The input is a candidate model coefficient. The output is a score telling us how badly that candidate fits the observed data.

This gives us two different graphs:

Graph	Horizontal axis	Vertical axis	Slope means
Prediction graph	Feature x	Predicted outcome $\hat{y}$	Change in predicted outcome per unit of feature
Error graph	Candidate coefficient β	Total squared error $S(\beta)$	Change in error per unit change in the coefficient

Confusing these two graphs is a common source of confusion. The derivative in the OLS derivation is the slope of the **error graph**, not the cable-cost slope being estimated.

5.0.2 Average change: the secant slope

Suppose coefficient $\beta = 2$ gives error $S(2) = 9$, while $\beta = 3$ gives error $S(3) = 4$. The average change in error is:

$$\frac{S(3) - S(2)}{3 - 2} = \frac{4 - 9}{1} = -5.$$

The negative sign says that, over this interval, increasing β reduced the error. This is a secant slope because it uses two different points on the error curve.

For a general small step h :

$$\text{secant slope} = \frac{S(\beta + h) - S(\beta)}{h}.$$

The letter h means a horizontal change. It might be 1, 0.1, 0.001, or another nonzero amount.

5.0.3 Instantaneous change: the tangent slope

To measure the slope at exactly one value of β , make h smaller and smaller. If the secant slopes approach a stable number, that number is the derivative:

$$\frac{dS}{d\beta} = \lim_{h \rightarrow 0} \frac{S(\beta + h) - S(\beta)}{h}$$

Read this as:

“The derivative of S with respect to β is the value approached by the change in S divided by the change in β as that change becomes extremely small.”

You do not divide by zero. Every secant calculation uses a nonzero h . The **limit** asks what value the ratios approach as h becomes arbitrarily small.

Geometrically, this derivative is the slope of the tangent line touching the error curve at the chosen β .

5.0.4 What the derivative's sign tells us

Value of $dS/d\beta$	Local direction of the error curve	What a small increase in β tends to do
Negative	Curve slopes downward	Decrease error
Zero	Tangent is horizontal	Produce no first-order change in error

Value of $dS/d\beta$	Local direction of the error curve	What a small increase in β tends to do
Positive	Curve slopes upward	Increase error

Imagine walking from left to right along a U-shaped valley:

- before the bottom, you walk downhill and the slope is negative;
- at the bottom, the ground is momentarily level and the slope is zero;
- after the bottom, you walk uphill and the slope is positive.

That is why a smooth minimum is found by solving:

$$\frac{dS}{d\beta} = 0.$$

5.0.5 Why “derivative zero” is not enough by itself

A horizontal tangent can occur at the bottom of a bowl, the top of a hill, or a flat pause on a curve. Such a point is called a **stationary point**.

We therefore inspect how the slope itself changes. This is the **second derivative**:

$$\frac{d^2S}{d\beta^2}.$$

Second derivative	Curvature	Typical stationary point
Positive	Bends upward, like a bowl	Minimum
Negative	Bends downward, like an upside-down bowl	Maximum
Zero	Inconclusive without more information	Could be flat or changing curvature

For OLS, the squared-error objective is a quadratic bowl. Its curvature cannot turn downward. This special structure is why setting the derivative to zero safely locates a global minimum once existence and rank are handled properly.

5.0.6 Derivative rules, proved from ordinary algebra

Only three elementary rules are needed for the one-parameter derivation.

Rule A: a constant has derivative zero If C never changes when β changes:

$$\frac{dC}{d\beta} = 0.$$

Using the difference quotient:

$$\frac{C - C}{h} = 0.$$

Rule B: the derivative of $c\beta$ is c For constant c :

$$\frac{c(\beta + h) - c\beta}{h} = \frac{ch}{h} = c.$$

So:

$$\frac{d}{d\beta}(c\beta) = c.$$

Rule C: the derivative of β^2 is 2β Start from the difference quotient:

$$\frac{(\beta + h)^2 - \beta^2}{h}.$$

Expand $(\beta + h)^2$:

$$\frac{\beta^2 + 2\beta h + h^2 - \beta^2}{h} = \frac{2\beta h + h^2}{h} = 2\beta + h.$$

As h approaches zero, $2\beta + h$ approaches 2β . Therefore:

$$\frac{d}{d\beta}(\beta^2) = 2\beta.$$

Multiplying by a constant c gives:

$$\frac{d}{d\beta}(c\beta^2) = 2c\beta.$$

These are not magical instructions. They are compressed results of the secant-to-tangent process.

5.0.7 From one direction to many: partial derivatives

With an intercept and one slope, the error has two inputs:

$$S(\beta_0, \beta_1).$$

You can imagine a landscape:

- east-west position represents the proposed intercept β_0 ;
- north-south position represents the proposed slope β_1 ; and
- elevation represents total squared error.

A **partial derivative** asks about one direction at a time:

$$\frac{\partial S}{\partial \beta_0}$$

means “change the intercept slightly while holding the slope fixed,” whereas:

$$\frac{\partial S}{\partial \beta_1}$$

means “change the slope slightly while holding the intercept fixed.”

We use the rounded symbol ∂ to signal that the function has several inputs and we are changing only one.

5.0.8 Gradient: all local slopes collected together

The **gradient** stacks all partial derivatives into a vector:

$$\nabla S(\beta_0, \beta_1) = \begin{bmatrix} \partial S / \partial \beta_0 \\ \partial S / \partial \beta_1 \end{bmatrix}.$$

The symbol ∇ is pronounced “nabla” or read simply as “the gradient.” The gradient points in the direction of steepest local increase in the error surface. Its negative points downhill most rapidly.

At the bottom of a smooth OLS bowl, neither a small intercept change nor a small slope change can reduce error. Both partial derivatives are zero:

$$\nabla S(\hat{\beta}_0, \hat{\beta}_1) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

With many features, the landscape has more dimensions than we can draw, but the rule is unchanged: one partial derivative per coefficient, all set to zero at an interior minimum.

5.0.9 The OLS search in one visual sequence

Calculus does not change the OLS goal. It turns “find the bottom” into equations that can be solved.

5.1 The scalar error bowl

Begin with a model that has one parameter and no intercept:

$$\hat{y}_i = \beta x_i.$$

The squared-error objective is:

$$S(\beta) = \sum_{i=1}^n (y_i - \beta x_i)^2.$$

Expand one squared term:

$$(y_i - \beta x_i)^2 = y_i^2 - 2\beta x_i y_i + \beta^2 x_i^2.$$

Therefore:

$$S(\beta) = \sum_i y_i^2 - 2\beta \sum_i x_i y_i + \beta^2 \sum_i x_i^2.$$

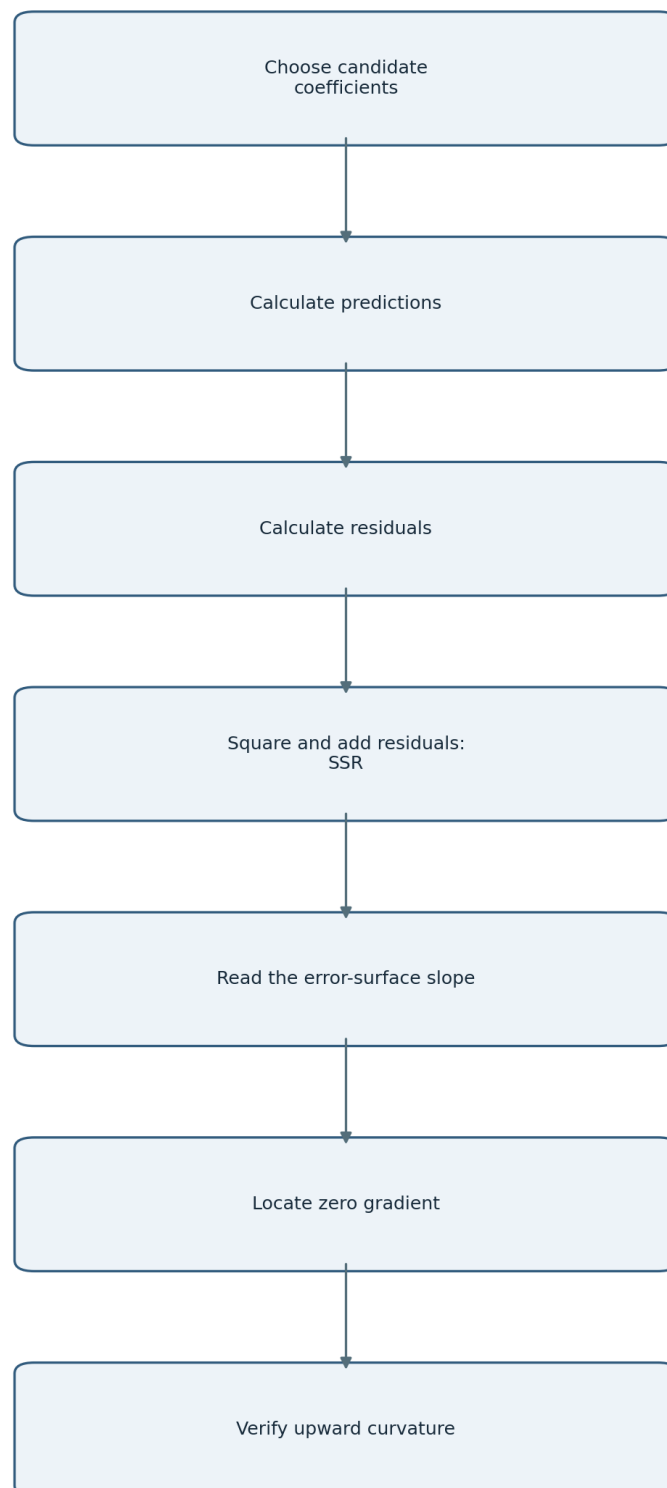
Notice what has happened. The observed data x_i and y_i are now fixed numbers. The only changing quantity is β . Therefore the three sums:

$$\sum_i y_i^2, \quad \sum_i x_i y_i, \quad \sum_i x_i^2$$

are constants. The objective has the general form:

$$S(\beta) = a\beta^2 + b\beta + c.$$

That is a **quadratic function**, whose graph is a parabola. Because:

**Figure 10:** Concept diagram

$$a = \sum_i x_i^2 \geq 0,$$

the parabola bends upward whenever at least one x_i is nonzero.

A complete numerical example

Let x represent cable length and y represent cost after removing a fixed baseline for this warm-up:

$$x = (1, 2, 3), \quad y = (2, 4, 7).$$

For a candidate $\beta = 2$:

Project	x_i	Actual y_i	Predicted $\hat{y}_i = 2x_i$	Residual $y_i - \hat{y}_i$	Squared residual
1	1	2	2	0	0
2	2	4	4	0	0
3	3	7	6	1	1

Thus $S(2) = 1$. For $\beta = 2.5$, the predictions are $(2.5, 5, 7.5)$ and:

$$S(2.5) = (-0.5)^2 + (-1)^2 + (-0.5)^2 = 1.5.$$

Trying values shows which candidate is better, but not yet which is best. The expanded objective gives:

$$\sum_i y_i^2 = 69, \quad \sum_i x_i y_i = 31, \quad \sum_i x_i^2 = 14,$$

so:

$$S(\beta) = 69 - 62\beta + 14\beta^2.$$

This equation describes the entire error bowl. Every possible through-origin slope β has exactly one height $S(\beta)$ on that bowl.

Differentiate with respect to β :

$$\frac{dS}{d\beta} = -2 \sum_i x_i y_i + 2\beta \sum_i x_i^2.$$

At a stationary point, the derivative is zero:

$$-2 \sum_i x_i y_i + 2\hat{\beta} \sum_i x_i^2 = 0.$$

Rearrange:

$$\hat{\beta} = \frac{\sum_i x_i y_i}{\sum_i x_i^2}.$$

The second derivative is:

$$\frac{d^2 S}{d\beta^2} = 2 \sum_i x_i^2.$$

If at least one x_i is nonzero, the second derivative is positive, so the stationary point is the unique minimum.

For the numerical example:

$$\hat{\beta} = \frac{31}{14} \approx 2.2143.$$

Its predictions are approximately (2.214, 4.429, 6.643), and its SSR is approximately 0.357. This is lower than both trial values:

$$S(2) = 1, \quad S(2.5) = 1.5.$$

Geometry of the derivative on the error curve

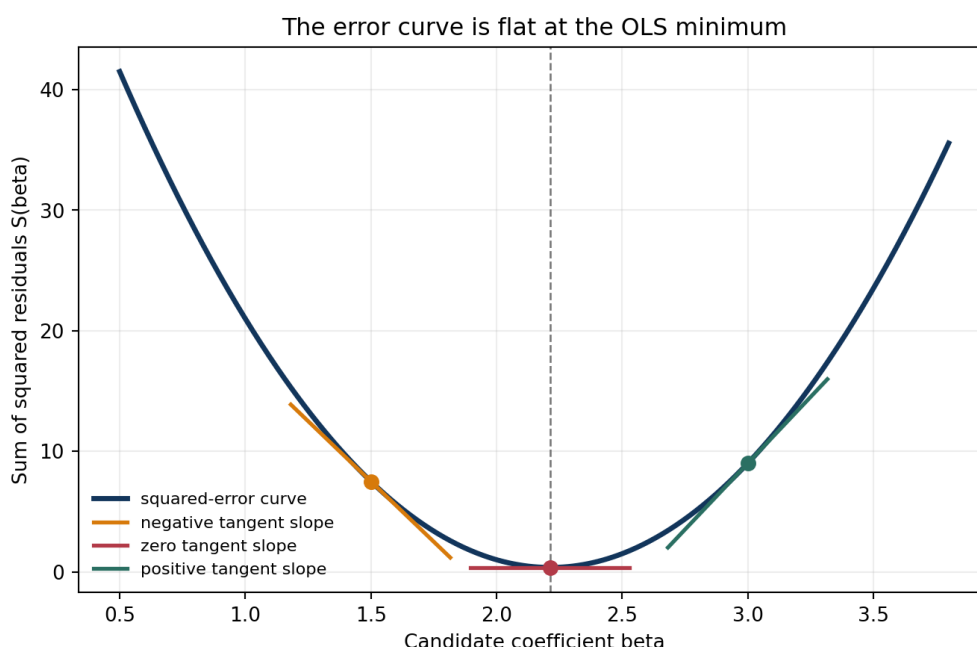


Figure 11: Negative, zero, and positive tangent slopes on the OLS squared-error curve.

At a candidate smaller than $\hat{\beta}$, the tangent to $S(\beta)$ slopes downward: increasing β reduces SSR. At a candidate larger than $\hat{\beta}$, it slopes upward: increasing β raises SSR. At $\hat{\beta}$, the tangent is horizontal.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 4.0, 7.0])

def S(beta):
    return np.sum((y - beta * x) ** 2)

def dS(beta):
    return -2 * np.sum(x * y) + 2 * beta * np.sum(x ** 2)
```

```

beta_hat = np.sum(x * y) / np.sum(x ** 2)
beta_grid = np.linspace(0.5, 3.8, 400)

plt.figure(figsize=(8, 5))
plt.plot(beta_grid, [S(b) for b in beta_grid], color="navy", label=r"$S(\beta)$")

for beta_point, colour in [(1.5, "darkorange"), (beta_hat, "firebrick"), (3.0, "darkgreen")]:
    tangent = S(beta_point) + dS(beta_point) * (beta_grid - beta_point)
    nearby = np.abs(beta_grid - beta_point) < 0.35
    plt.plot(beta_grid[nearby], tangent[nearby], color=colour)
    plt.scatter(beta_point, S(beta_point), color=colour, zorder=3)

plt.axvline(beta_hat, color="grey", linestyle="--", alpha=0.6)
plt.xlabel(r"Candidate coefficient $\beta$")
plt.ylabel("Sum of squared residuals")
plt.title("Tangents point downhill before the minimum and uphill after it")
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()

```

The three tangent lines make the derivative visible. The derivative is not the height of the curve. It is the curve's local slope.

Code proof: derivative, grid search, and analytic solution

```

import numpy as np

x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 4.0, 7.0])

def ssr_through_origin(beta, x, y):
    return np.sum((y - beta * x) ** 2)

def derivative_through_origin(beta, x, y):
    return -2.0 * np.sum(x * y) + 2.0 * beta * np.sum(x ** 2)

beta_hat = np.sum(x * y) / np.sum(x ** 2)
grid = np.linspace(0.0, 4.0, 4001)
grid_best = grid[np.argmin([ssr_through_origin(b, x, y) for b in grid])]

print("Analytic minimum:", beta_hat)
print("Grid minimum:      ", grid_best)
print("Derivative there:", derivative_through_origin(beta_hat, x, y))

assert np.isclose(derivative_through_origin(beta_hat, x, y), 0.0)
assert np.isclose(beta_hat, grid_best, atol=0.001)
assert 2.0 * np.sum(x ** 2) > 0.0

```

The grid is not how we want to fit the model; it is an independent numerical check that the analytic solution sits at the bottom of the bowl.

5.2 One feature with an intercept

Now use:

$$\hat{y}_i = \beta_0 + \beta_1 x_i.$$

The objective is:

$$S(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2.$$

There are two unknown parameters, so there are two partial derivatives.

For the intercept:

$$\frac{\partial S}{\partial \beta_0} = -2 \sum_i (y_i - \beta_0 - \beta_1 x_i).$$

Set it to zero:

$$\sum_i y_i - n\hat{\beta}_0 - \hat{\beta}_1 \sum_i x_i = 0.$$

Divide by n :

$$\bar{y} - \hat{\beta}_0 - \hat{\beta}_1 \bar{x} = 0,$$

so:

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

This proves that the fitted OLS line with an intercept passes through $(\bar{x}, \bar{y})$.

The result has a direct geometric meaning. The point $(\bar{x}, \bar{y})$ is the **centre of balance** of the observed point cloud: the mean horizontal position and the mean vertical position. When an intercept is included, the OLS line is anchored through this centre exactly, not approximately.

For the slope:

$$\frac{\partial S}{\partial \beta_1} = -2 \sum_i x_i (y_i - \beta_0 - \beta_1 x_i).$$

Substituting the intercept relationship and simplifying gives:

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}.$$

The numerator is the unnormalised co-movement of x and y ; the denominator is the unnormalised variation in x . If every x_i is identical, the denominator is zero and the slope cannot be identified.

Reading the slope formula from the ground up

The notation $x_i - \bar{x}$ asks how far project i lies to the right or left of the average project on the x -axis. Similarly, $y_i - \bar{y}$ asks how far it lies above or below the average on the y -axis.

For each project, multiply those two deviations:

- right of average and above average: positive times positive, giving a positive product;
- left of average and below average: negative times negative, also giving a positive product;

- right of average but below average: positive times negative, giving a negative product; and
- left of average but above average: negative times positive, also giving a negative product.

Therefore the numerator:

$$\sum_i (x_i - \bar{x})(y_i - \bar{y})$$

is positive when x and y generally move together, negative when they generally move in opposite directions, and close to zero when their linear co-movement is weak.

The denominator:

$$\sum_i (x_i - \bar{x})^2$$

measures the horizontal spread of the data. Dividing by that spread converts total co-movement into a change in y per unit of x .

Hand calculation with four projects

Use:

$$x = (1, 2, 3, 4), \quad y = (12, 19, 29, 38).$$

The means are:

$$\bar{x} = 2.5, \quad \bar{y} = 24.5.$$

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	Product	Squared x deviation
1	12	-1.5	-12.5	18.75	2.25
2	19	-0.5	-5.5	2.75	0.25
3	29	0.5	4.5	2.25	0.25
4	38	1.5	13.5	20.25	2.25
Sum				44	5

Thus:

$$\hat{\beta}_1 = \frac{44}{5} = 8.8,$$

and:

$$\hat{\beta}_0 = 24.5 - (8.8)(2.5) = 2.5.$$

The fitted line is:

$$\boxed{\hat{y} = 2.5 + 8.8x}.$$

The slope has units of “units of y per unit of x .” If x is cable kilometres and y is million PKR, the fitted association is 8.8 million PKR per additional kilometre in this small dataset.

Why identical x -values make slope impossible, not merely unstable

Suppose:

$$x = (5, 5, 5), \quad y = (10, 12, 14).$$

Then $\bar{x} = 5$, so every horizontal deviation is zero. The data form a vertical stack of points at one observed x -position. They contain no horizontal movement from which a rise-to-run ratio could be learned.

Every pair satisfying:

$$\beta_0 + 5\beta_1 = 12$$

produces the same prediction, 12, for all three observed projects. For example:

$$(\beta_0, \beta_1) = (12, 0), \quad (7, 1), \quad (2, 2)$$

all fit the observed $x = 5$ position identically. The question “how does y change when x changes?” is unanswered because x did not change. This is lack of identification in the data, not a defect that more decimal precision can repair.

Geographical interpretation: If every observed project comes from the same accessibility category, the data cannot estimate how cost changes across accessibility categories. A detailed map cannot substitute for variation in the feature relevant to the estimand.

Code proof: centred formula, intercept, and mean point

```
x = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([12.0, 19.0, 29.0, 38.0])

x_centered = x - x.mean()
y_centered = y - y.mean()

beta_1 = np.sum(x_centered * y_centered) / np.sum(x_centered ** 2)
beta_0 = y.mean() - beta_1 * x.mean()

print("beta_0:", beta_0)
print("beta_1:", beta_1)

# The fitted line passes through ( $\bar{x}$ ,  $\bar{y}$ )
assert np.isclose(beta_0 + beta_1 * x.mean(), y.mean())

# Both partial derivatives are zero
residuals = y - (beta_0 + beta_1 * x)
partial_b0 = -2.0 * np.sum(residuals)
partial_b1 = -2.0 * np.sum(x * residuals)
assert np.isclose(partial_b0, 0.0, atol=1e-10)
assert np.isclose(partial_b1, 0.0, atol=1e-10)
```

5.3 The matrix objective

For multiple features, collect all parameters in β and write:

$$S(\beta) = (y - X\beta)^T(y - X\beta).$$

Expand carefully:

$$\begin{aligned} S(\beta) &= (y^T - \beta^T X^T)(y - X\beta) \\ &= y^T y - y^T X\beta - \beta^T X^T y + \beta^T X^T X\beta. \end{aligned}$$

The middle two terms are scalars and transposes of one another, so they are equal:

$$y^T X\beta = (y^T X\beta)^T = \beta^T X^T y.$$

Therefore:

$$S(\beta) = y^T y - 2\beta^T X^T y + \beta^T X^T X\beta.$$

5.4 The matrix derivative without hand-waving

A numeric 2×2 quadratic before index notation

Let

$$A = \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_1 \\ \beta_2 \end{bmatrix}.$$

Writing all four double-sum terms gives

$$\beta^T A \beta = 2\beta_1^2 + \beta_1\beta_2 + 3\beta_2\beta_1 + 4\beta_2^2 = 2\beta_1^2 + 4\beta_1\beta_2 + 4\beta_2^2.$$

Differentiate that ordinary polynomial one coordinate at a time:

$$\frac{\partial}{\partial \beta_1} = 4\beta_1 + 4\beta_2, \quad \frac{\partial}{\partial \beta_2} = 4\beta_1 + 8\beta_2.$$

At $\beta = (5, 6)^T$, the gradient is $(44, 68)^T$. Now verify that the matrix rule produces the same result:

```
import numpy as np

A = np.array([[2.0, 1.0], [3.0, 4.0]])
beta = np.array([5.0, 6.0])
gradient_by_expansion = np.array([
    4 * beta[0] + 4 * beta[1],
    4 * beta[0] + 8 * beta[1],
])
gradient_by_matrix_rule = (A + A.T) @ beta

print(gradient_by_expansion)
assert np.allclose(gradient_by_expansion, gradient_by_matrix_rule)
```

The general argument below says exactly the same thing for any matrix size.

Three derivative facts are needed.

First, $y^T y$ contains no β , so:

$$\nabla_{\beta}(y^T y) = 0.$$

Second, for a constant vector c :

$$\nabla_{\beta}(\beta^T c) = c.$$

Thus:

$$\nabla_{\beta}(-2\beta^T X^T y) = -2X^T y.$$

Third, let $A = X^T X$. Expand the quadratic form by coordinates:

$$\beta^T A \beta = \sum_j \sum_k \beta_j A_{jk} \beta_k.$$

The derivative with respect to coordinate β_r collects terms where $j = r$ and where $k = r$:

$$\frac{\partial}{\partial \beta_r}(\beta^T A \beta) = \sum_k A_{rk} \beta_k + \sum_j \beta_j A_{jr}.$$

In vector form:

$$\nabla_{\beta}(\beta^T A \beta) = (A + A^T)\beta.$$

Because $A = X^T X$ is symmetric, $A = A^T$:

$$\nabla_{\beta}(\beta^T X^T X \beta) = 2X^T X \beta.$$

Combine the three derivatives:

$$\nabla_{\beta} S(\beta) = -2X^T y + 2X^T X \beta.$$

At the minimum:

$$-2X^T y + 2X^T X \hat{\beta} = 0,$$

which reduces again to:

$$X^T X \hat{\beta} = X^T y.$$

Code proof: compare the analytic gradient with finite differences

A finite-difference gradient estimates each partial derivative by nudging one parameter a very small distance:

$$\frac{\partial S}{\partial \beta_j} \approx \frac{S(\beta + h u_j) - S(\beta - h u_j)}{2h},$$

where u_j is a vector with 1 in position j and 0 elsewhere.

```
import numpy as np

X = np.array([
    [1.0, 1.0],
    [1.0, 2.0],
    [1.0, 3.0],
    [1.0, 4.0],
])
y = np.array([12.0, 19.0, 29.0, 38.0])
beta = np.array([4.0, 7.0])
```

```

def ssr(beta_vector):
    residuals = y - X @ beta_vector
    return residuals @ residuals

analytic_gradient = -2.0 * X.T @ (y - X @ beta)

h = 1e-6
finite_difference_gradient = np.zeros_like(beta)
for j in range(beta.size):
    unit = np.zeros_like(beta)
    unit[j] = 1.0
    finite_difference_gradient[j] = (
        ssr(beta + h * unit) - ssr(beta - h * unit)
    ) / (2.0 * h)

print("Analytic:", analytic_gradient)
print("Finite difference:", finite_difference_gradient)
assert np.allclose(analytic_gradient, finite_difference_gradient, rtol=1e-6)

```

The finite-difference calculation is an independent numerical audit of the matrix derivative. A poor choice of h can introduce approximation or floating-point error; its role is verification, not fitting.

5.5 Isolating $\hat{\beta}$ and stating the missing condition

If $X^T X$ is invertible:

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

This famous closed form is valid only when the design matrix has full column rank. In plain language, no design column can be constructed exactly from the others, and there must be enough independent information to estimate every parameter.

The Hessian—the matrix of second derivatives—is:

$$\nabla_{\beta}^2 S(\beta) = 2X^T X.$$

For any vector z :

$$z^T X^T X z = (Xz)^T (Xz) = \|Xz\|_2^2 \geq 0.$$

Thus $X^T X$ is positive semidefinite and the objective is convex. If X has full column rank, $Xz \neq 0$ for every nonzero z , so the quantity is strictly positive and the objective is strictly convex. The minimiser is then unique.

Translate the Hessian statement into a picture

The **Hessian** is the many-parameter version of the second derivative. It answers:

If we move the coefficient vector in some direction, does the error surface curve upward, downward, or remain flat in that direction?

The test vector z represents any proposed direction of movement in coefficient space. The quantity:

$$z^T X^T X z = \|Xz\|_2^2$$

is a squared length. It can be positive or zero but never negative. Therefore the OLS error surface cannot bend downward in any direction.

This is the meaning of **positive semidefinite**:

$$z^T A z \geq 0 \quad \text{for every direction } z.$$

The word **semi** allows equality with zero. A zero-curvature direction creates a flat valley: one can move through coefficient space without changing predictions or SSR. That is what happens under exact multicollinearity.

If X has full column rank, every nonzero coefficient movement changes the fitted predictions, so:

$$\|Xz\|_2^2 > 0 \quad \text{for every } z \neq 0.$$

Now the Hessian is **positive definite**, the bowl has no flat direction, and the coefficient vector at the bottom is unique.

Convexity and the global minimum

A **local minimum** is lower than nearby points. A **global minimum** is lower than every point on the entire surface.

An irregular mountain landscape can contain many local valleys separated by ridges. OLS does not have that shape. Its convex error surface is bowl-shaped: the straight segment between any two points on the surface never passes below the surface itself. Consequently, a zero-gradient point cannot be a small local valley with a deeper valley hidden elsewhere. It is a global minimum.

The bowl may have:

- one bottom point when X has full column rank; or
- a flat bottom valley when rank fails.

In both cases, least-squares fitted predictions exist. Full rank is needed for a **unique coefficient vector** and for the ordinary inverse formula, not for the mere existence of a least-squares minimum.

Code proof: positive semidefiniteness

```
rng = np.random.default_rng(7)
X = rng.normal(size=(20, 4))
z = rng.normal(size=4)

left = z.T @ X.T @ X @ z
right = np.linalg.norm(X @ z) ** 2

print(left, right)
assert np.isclose(left, right)
assert left >= -1e-12 # tolerance for floating-point roundoff
```

5.6 Why we do not compute the inverse in application code

The derivation uses $(X^T X)^{-1}$ to express the mathematical solution. A program should usually solve:

$$(X^T X) \hat{\beta} = X^T y$$

with:

```
beta_hat = np.linalg.solve(X.T @ X, X.T @ y)
```

Better still for OLS, solve the least-squares problem directly:

```
beta_hat, residual_sums, rank, singular_values = np.linalg.lstsq(X, y, rcond=None)
```

Why?

- Forming $X^T X$ squares the condition number and can magnify numerical instability.
- Explicit inversion performs unnecessary work.
- `lstsq` uses a decomposition designed for least-squares problems and also reports rank and singular values.

This is not a conceptual shortcut. We have already built and derived the objective. It is a numerical implementation choice that respects the same mathematics.

Research note: fitting assumptions are not inference assumptions

The OLS coefficient calculation does **not** require the residuals to be normally distributed. To compute a least-squares projection, we need a defined numerical problem; for a unique coefficient vector, we additionally need full column rank.

Stronger claims require stronger assumptions. Unbiased causal or explanatory coefficients require a defensible relationship between included features and the unobserved error. Classical small-sample confidence intervals add assumptions about variance and, in some treatments, normality. Consistency, robust standard errors, clustered data, and sampling design introduce further conditions.

Keep three layers separate:

1. **Algebra:** Did the program minimise SSR correctly?
2. **Statistical generalisation:** Does this sample support claims about new projects or a wider population?
3. **Causal interpretation:** Does the design identify an intervention effect?

A perfect answer at the first layer does not settle the next two.

5.7 Rank failure: two columns carrying the same information

Suppose the design matrix includes both cable length in kilometres and the same cable length in metres:

$$x_{\text{metres}} = 1000x_{\text{kilometres}}$$

One column is an exact multiple of another. The model cannot decide how much coefficient to assign to each, because infinitely many pairs produce the same combined prediction.

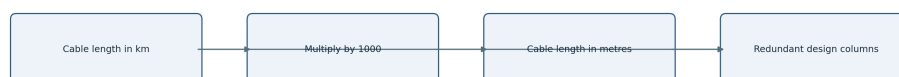


Figure 12: Concept diagram

Figure 5.1 — Perfect multicollinearity. More columns do not necessarily provide more information.

Break it deliberately

```
x_km = np.array([1.0, 2.0, 3.0, 4.0])
x_m = 1000.0 * x_km
y = np.array([12.0, 19.0, 29.0, 38.0])

X_bad = np.column_stack([np.ones(x_km.size), x_km, x_m])

print("columns:", X_bad.shape[1])
print("rank:    ", np.linalg.matrix_rank(X_bad))

try:
    beta_bad = np.linalg.solve(X_bad.T @ X_bad, X_bad.T @ y)
except np.linalg.LinAlgError as error:
    print("Normal-equation solve failed:", error)

beta_lstsq, _, rank, singular_values = np.linalg.lstsq(X_bad, y, rcond=None)
print("One least-squares solution:", beta_lstsq)
print("Reported rank:", rank)
print("Singular values:", singular_values)
```

lstsq can return a prediction-minimising solution, but the individual coefficients on kilometres and metres are not separately meaningful. A numerical answer does not repair an unidentified interpretation.

The correct response is usually to remove the redundant feature or redesign the representation—not to celebrate that the program returned numbers.

5.8 Figure lab: see the error valley

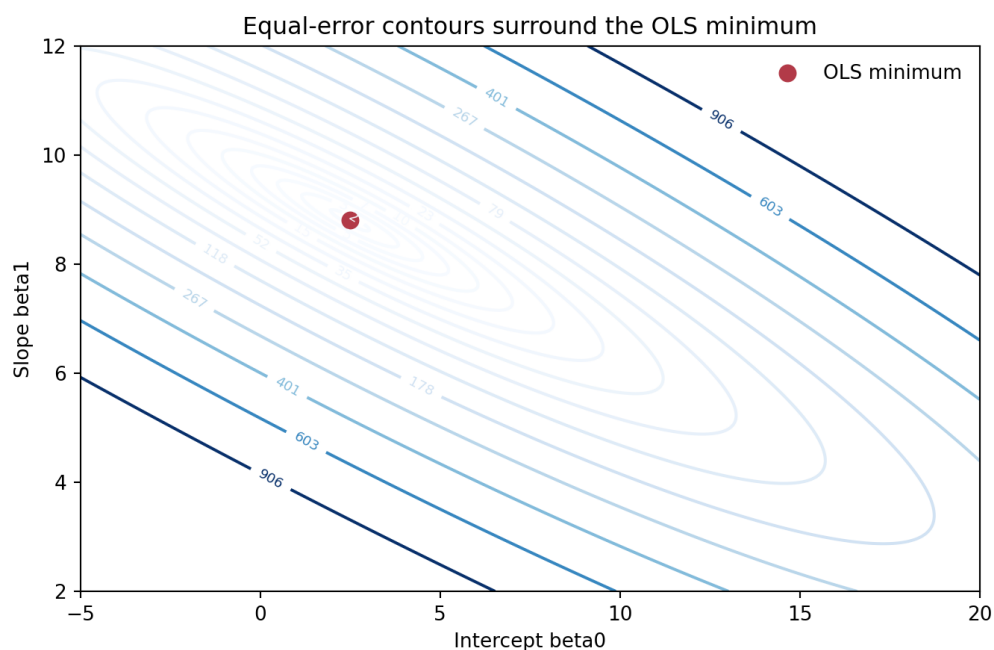


Figure 13: Contours of equal squared error surrounding the OLS minimum.

For a model with an intercept and one slope, SSR is a surface over (β_0, β_1) .

```
import matplotlib.pyplot as plt
import numpy as np
```

```

x = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([12.0, 19.0, 29.0, 38.0])

b0_grid = np.linspace(-5.0, 20.0, 200)
b1_grid = np.linspace(2.0, 12.0, 200)
B0, B1 = np.meshgrid(b0_grid, b1_grid)

SSR = np.zeros_like(B0)
for i in range(x.size):
    SSR += (y[i] - (B0 + B1 * x[i])) ** 2

X = np.column_stack([np.ones(x.size), x])
beta_hat = np.linalg.lstsq(X, y, rcond=None)[0]

plt.figure(figsize=(8, 6))
contours = plt.contour(B0, B1, SSR, levels=25)
plt.clabel(contours, inline=True, fontsize=7)
plt.scatter(beta_hat[0], beta_hat[1], color="red", s=80, label="OLS minimum")
plt.xlabel("Intercept  $\beta_0$ ")
plt.ylabel("Slope  $\beta_1$ ")
plt.title("Contours of the OLS error surface")
plt.legend()
plt.tight_layout()
plt.show()

```

Figure 5.2 — The OLS error valley. Each contour joins parameter pairs with equal SSR. The centre is the minimum. With highly correlated features, the contours become long and narrow; with perfect collinearity, the unique centre becomes a flat valley.

5.9 Build: the complete Week 1 estimator

The complete application below consolidates the daily increments. Replace the partial `mhp_estimator.py` with this version.

```

from dataclasses import dataclass
from typing import Literal, Sequence

import numpy as np

AnalysisPurpose = Literal["prediction", "explanation", "causation"]

@dataclass(frozen=True)
class AnalysisContract:
    purpose: AnalysisPurpose
    target_name: str
    target_unit: str
    unit_of_observation: str
    intended_use: str

    def __post_init__(self):
        allowed = {"prediction", "explanation", "causation"}
        if self.purpose not in allowed:
            raise ValueError(f"purpose must be one of {sorted(allowed)}")
        for name, value in vars(self).items():
            if not str(value).strip():
                raise ValueError(f"{name} cannot be empty")

```

```

class MHPCostEstimator:
    """Small OLS estimator for learning, diagnostics, and explicit reasoning."""

    def __init__(self, contract: AnalysisContract):
        self.contract = contract
        self.parameters_ = None
        self.feature_names_ = None
        self.rank_ = None
        self.singular_values_ = None
        self.training_metrics_ = None
        self.is_fitted_ = False

    def __repr__(self):
        status = "fitted" if self.is_fitted_ else "unfitted"
        return f"MHPCostEstimator(purpose={self.contract.purpose!r}, {status})"

    @staticmethod
    def _validate_inputs(X, y=None):
        X = np.asarray(X, dtype=float)
        if X.ndim == 1:
            X = X.reshape(-1, 1)
        if X.ndim != 2:
            raise ValueError(f"X must be 2D after conversion; got shape {X.shape}")
        if X.shape[0] == 0 or X.shape[1] == 0:
            raise ValueError("X must contain at least one row and one feature")
        if not np.isfinite(X).all():
            raise ValueError("X contains NaN or infinite values")

        if y is None:
            return X

        y = np.asarray(y, dtype=float)
        if y.ndim == 2 and y.shape[1] == 1:
            y = y.reshape(-1)
        if y.ndim != 1:
            raise ValueError(f"y must be 1D; got shape {y.shape}")
        if X.shape[0] != y.shape[0]:
            raise ValueError(
                f"row mismatch: X has {X.shape[0]} rows but y has {y.shape[0]}"
            )
        if not np.isfinite(y).all():
            raise ValueError("y contains NaN or infinite values")
        return X, y

    @staticmethod
    def _add_intercept_column(X):
        return np.column_stack([np.ones(X.shape[0]), X])

    @staticmethod
    def _metrics_from_predictions(y, y_hat):
        y = np.asarray(y, dtype=float).reshape(-1)
        y_hat = np.asarray(y_hat, dtype=float).reshape(-1)
        if y.shape != y_hat.shape:
            raise ValueError("y and y_hat must have identical shapes")

        residuals = y - y_hat

```



```

ssr = float(residuals @ residuals)
mse = ssr / residuals.size
return {
    "ssr": ssr,
    "mse": mse,
    "rmse": float(np.sqrt(mse)),
    "mae": float(np.mean(np.abs(residuals))),
}

def fit(self, X, y, feature_names: Sequence[str]):
    X, y = self._validate_inputs(X, y)
    feature_names = [str(name).strip() for name in feature_names]

    if len(feature_names) != X.shape[1]:
        raise ValueError(
            f"received {len(feature_names)} names for {X.shape[1]} features"
        )
    if any(not name for name in feature_names):
        raise ValueError("feature names cannot be empty")
    if len(set(feature_names)) != len(feature_names):
        raise ValueError("feature names must be unique")

    X_design = self._add_intercept_column(X)
    parameters, _, rank, singular_values = np.linalg.lstsq(
        X_design, y, rcond=None
    )

    required_rank = X_design.shape[1]
    if rank < required_rank:
        raise ValueError(
            "design matrix is rank deficient: "
            f"rank {rank}, but {required_rank} independent columns are required"
        )

    self.parameters_ = parameters
    self.feature_names_ = feature_names
    self.rank_ = int(rank)
    self.singular_values_ = singular_values
    self.is_fitted_ = True

    training_predictions = X_design @ self.parameters_
    self.training_metrics_ = self._metrics_from_predictions(
        y, training_predictions
    )
    return self

def predict(self, X):
    if not self.is_fitted_:
        raise RuntimeError("fit the estimator before calling predict")

    X = np.asarray(X, dtype=float)
    if X.ndim == 1:
        X = X.reshape(1, -1)
    X = self._validate_inputs(X)
    if X.shape[1] != len(self.feature_names_):
        raise ValueError(
            f"expected {len(self.feature_names_)} features, got {X.shape[1]}"
        )

```

```

        return self._add_intercept_column(X) @ self.parameters_

    def evaluate(self, X, y):
        X, y = self._validate_inputs(X, y)
        return self._metrics_from_predictions(y, self.predict(X))

    def coefficients(self):
        if not self.is_fitted_:
            raise RuntimeError("fit the estimator before requesting coefficients")
        names = ["intercept", *self.feature_names_]
        return dict(zip(names, self.parameters_))

```

5.10 End-to-end use on synthetic KP projects

The small dataset below has three pre-construction features. Keeping it small makes the entire calculation inspectable; it is not enough for a real deployment.

```

import numpy as np

contract = AnalysisContract(
    purpose="prediction",
    target_name="actual_cost_m_pkr",
    target_unit="million PKR",
    unit_of_observation="one completed MHP project",
    intended_use="design-stage budget review for approved projects",
)

# Columns: road distance (km), planned capacity (kW), terrain index (1–5)
X_train = np.array([
    [5.0, 80.0, 2.0],
    [8.0, 100.0, 2.0],
    [12.0, 120.0, 3.0],
    [15.0, 150.0, 3.0],
    [18.0, 160.0, 4.0],
    [22.0, 180.0, 4.0],
    [25.0, 200.0, 5.0],
    [30.0, 240.0, 5.0],
])

y_train = np.array([19.0, 23.0, 30.0, 35.0, 42.0, 47.0, 55.0, 63.0])

model = MHPCostEstimator(contract).fit(
    X_train,
    y_train,
    feature_names=["road_distance_km", "planned_capacity_kw", "terrain_index"],
)

print(model)
print("Coefficients:", model.coefficients())
print("Training metrics:", model.training_metrics_)

X_new = np.array([
    [20.0, 170.0, 4.0],
    [10.0, 110.0, 3.0],
])

print("New predictions:", model.predict(X_new))

```

Interpretation discipline

If the fitted road-distance coefficient is positive, the prediction reading is:

“Holding the other included numerical features fixed, the fitted rule adds this many million PKR for each additional kilometre of recorded road distance.”

It is **not** automatically:

“Building one kilometre of road will change MHP cost by this amount.”

The second statement is causal and may even reverse the intervention: the feature is distance from a road, while the action is constructing a road.

Also inspect whether “holding capacity and terrain fixed” describes plausible comparisons in the dataset. Regression can calculate contrasts that the observed projects barely support.

5.11 Verify the class against the equations

```
X_design = np.column_stack([np.ones(X_train.shape[0]), X_train])
beta_from_normal_equations = np.linalg.solve(
    X_design.T @ X_design,
    X_design.T @ y_train,
)

assert np.allclose(model.parameters_, beta_from_normal_equations)

residuals = y_train - model.predict(X_train)
assert np.allclose(X_design.T @ residuals, 0.0, atol=1e-9)
assert np.isclose(residuals.sum(), 0.0, atol=1e-9)

metrics = model.evaluate(X_train, y_train)
assert np.isclose(metrics["ssr"], residuals @ residuals)
assert np.isclose(metrics["rmse"], np.sqrt(np.mean(residuals ** 2)))
```

These are not decorative tests. Each one ties an application output to a mathematical result developed earlier in the chapter.

5.12 What the completed application still cannot do

The Week 1 estimator is a learning instrument, not a deployment-ready public investment system. It does not yet:

- split data into training, validation, and test sets;
- handle categorical districts, project types, or contractors;
- standardise features or examine conditioning;
- quantify coefficient uncertainty;
- diagnose heteroskedasticity, nonlinearity, or influential observations;
- model time trends, inflation, or district-level clustering;
- protect against data leakage across project phases;
- establish causal effects;
- document data lineage, permissions, or revision history; or
- monitor performance when construction conditions change.

Recognising these limits is part of mastery.

5.13 Concept checkpoint: explain before coding

Answer these in ordinary language before returning to the implementation:

1. What is the difference between the slope of the fitted line $\hat{y} = \beta_0 + \beta_1 x$ and the derivative $dS/d\beta$ of the error curve?
2. How does a secant line lead us to the idea of a tangent line? Why is a tangent a local description rather than a description of the whole curve?
3. In the formula $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$, why does the fitted line pass through $(\bar{x}, \bar{y})$? Why does a zero denominator in the slope formula make the slope unidentifiable rather than merely numerically unstable?
4. Why are $y^T X \beta$ and $\beta^T X^T y$ guaranteed to be equal? State both the transpose rule and the scalar rule being used.
5. What does positive semidefinite mean? Explain why it gives OLS a bowl or flat-valley shape and prevents a higher local minimum from hiding away from a lower global minimum.
6. When cable length in kilometres and metres both appear in X , `lstsq` can still return numbers. Why are the separate coefficients not identified by the data?
7. Why does an algebraically correct OLS solution tell us nothing by itself about whether cable length causes project cost to rise?
8. Give one example of a geographical mechanism that could make a slope estimated in accessible valleys misleading for remote valleys.

5.14 Day 5 practice

1. Derive the through-origin slope without looking at the text.
2. Explain, by coordinates, why the derivative of $\beta^T A \beta$ is $(A + A^T)\beta$.
3. Construct a full-rank matrix with four rows and three columns. Confirm its rank in NumPy.
4. Make one column an exact combination of the other columns. Confirm the rank loss.
5. Change one target in the end-to-end dataset by 20 million PKR. Compare coefficient changes, RMSE, and MAE.
6. Fit the same data after converting road distance from kilometres to metres. Verify that predictions are unchanged and explain the coefficient change.
7. Write a one-paragraph briefing for a senior manager that reports a prediction, its unit, its intended use, and two limitations without using mathematical jargon.

Day 5 exit check

You are ready to leave Week 1 when you can move in both directions:

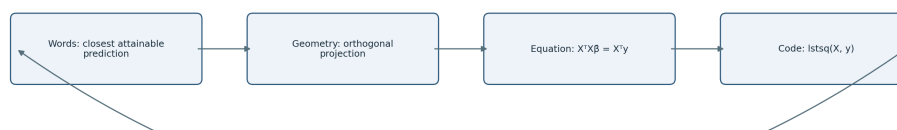


Figure 14: Concept diagram

Week 1 Capstone — Build It, Break It, Rebuild It

Capstone brief

You are given 40 synthetic completed MHP projects and asked to create an early-stage cost estimator. Complete the work in four passes.

Pass 1: define

Write the AnalysisContract. State:

- the unit of observation;
- the target and price basis;
- the point in the project cycle when prediction occurs;
- the decision supported; and
- whether the work is prediction, explanation, or causation.

Pass 2: build

Use at least three features known at the stated prediction time. Fit the model, report coefficients with units, and calculate training metrics.

Pass 3: break

Create and explain each failure:

1. a row-alignment error;
2. a missing or infinite value;
3. a redundant feature;
4. a feature recorded after construction begins; and
5. one extreme target value.

Pass 4: rebuild

Correct the failures. Add a held-out set of projects. Compare training and held-out RMSE and MAE. Write a recommendation that separates what the model can support from what it cannot.

Assessment rubric

Dimension	Emerging	Competent	Mastery
Question	Purpose is vague	Purpose and target are stated	Estimand, timing, and intended use are precise
Mathematics	Formula copied	Main steps reproduced	Derivation explained and verified independently
Code	Runs once	Validates common failures	Tests equations, shapes, rank, and interpretation boundaries
Geometry	Terms repeated	Projection described	Column space, orthogonality, and minimum distance connected
Context	Generic example	KP units and features used	Institutional consequences and data limitations addressed
Communication	Numbers listed	Metrics interpreted	Decision, uncertainty, and limits clearly separated

Formula Sheet

Concept	Equation	Meaning
Linear prediction	$\hat{y} = X\beta$	Weighted combination of design columns
Residual	$e = y - \hat{y}$	Actual minus predicted
SSR	$e^T e = \sum_i e_i^2$	Total squared training error
MSE	SSR/n	Mean squared error
RMSE	$\sqrt{SSR/n}$	Squared-error summary in target units
MAE	$\frac{1}{n} \sum_i e_i $	Mean absolute miss
Orthogonality	$X^T e = 0$	Residual perpendicular to every design column
Normal equations	$X^T X \hat{\beta} = X^T y$	First-order condition for OLS
Closed form	$(X^T X)^{-1} X^T y$	Unique solution when X has full column rank
Scalar slope	$\frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$	One-feature slope with intercept
Scalar intercept	$\bar{y} - \hat{\beta}_1 \bar{x}$	Makes the line pass through the means
Hessian	$2X^T X$	Curvature of the SSR surface

Glossary

Association: A numerical relationship in observed data; it need not be causal.

Column space: All vectors that can be formed as weighted combinations of a matrix's columns.

Confounder: A common cause that can distort an exposure–outcome association.

Counterfactual: The outcome that would have occurred under an alternative action or condition.

Design matrix: The rectangular matrix of features used to construct model predictions.

Estimator: A rule or algorithm that converts data into parameter estimates.

Feature: An input variable used by a model.

Full column rank: A condition in which no design column is an exact linear combination of the others.

Gradient: A vector of partial derivatives; it points in the direction of steepest local increase.

Hessian: A matrix of second derivatives describing local curvature.

Hyperparameter: A modeling setting chosen rather than estimated by the fitting calculation.

Intercept: The constant term added to every linear prediction.

Loss function: A numerical rule that penalises prediction error during fitting.

Multicollinearity: Strong linear dependence among features; perfect multicollinearity

causes rank deficiency.

Observation: One unit represented by one row of the dataset.

OLS: Ordinary least squares, the estimator that minimises the sum of squared residuals.

Parameter: A numerical value learned from data by an estimator.

Projection: The closest point in a subspace to a given vector under Euclidean distance.

Rank: The number of linearly independent directions represented by a matrix.

Residual: The observed target minus its fitted prediction.

Target: The outcome the model estimates.

Target leakage: Use of information that would not legitimately be available at prediction time.

Instructor and Self-Study Notes

Suggested daily timebox

Activity	Minutes
Orient and retrieve prior knowledge	15
Work through the central derivation	35
Run and alter the proof code	35
Create or inspect the figure	20
Extend the application	35
Break-and-repair exercise	25
Exit check and written reflection	15
Total	180

Three hours is a guide, not a race. If a learner cannot explain yesterday's exit check, begin there rather than adding more notation.

Retrieval prompts for the next morning

- After Day 1: "Name the three regression jobs and one risk of each."
- After Day 2: "Draw the shapes of X , y , and β ."
- After Day 3: "Expand one row of $X\beta$ and explain the ones column."
- After Day 4: "Why is the OLS residual perpendicular to X ?"
- After Day 5: "State the normal equations and the condition for a unique coefficient vector."

Guidance on the KP context

Context should discipline the mathematics, not decorate it. Ask repeatedly:

- At what project stage is this field known?
- Who recorded it and for what administrative purpose?
- Does one row mean the same thing across districts and implementing partners?
- Are costs expressed in the same price year?
- Does a terrain index of 4 mean the same thing to every assessor?
- Could the model penalise remote projects for structural conditions outside field teams' control?
- Would an apparent "efficiency" relationship survive a fair comparison of project types?

These questions connect modeling capability to institutional responsibility.

Where Week 2 Begins

Week 1 ends with a correct small OLS implementation and an honest account of its limits. Week 2 should extend the same application in this order:

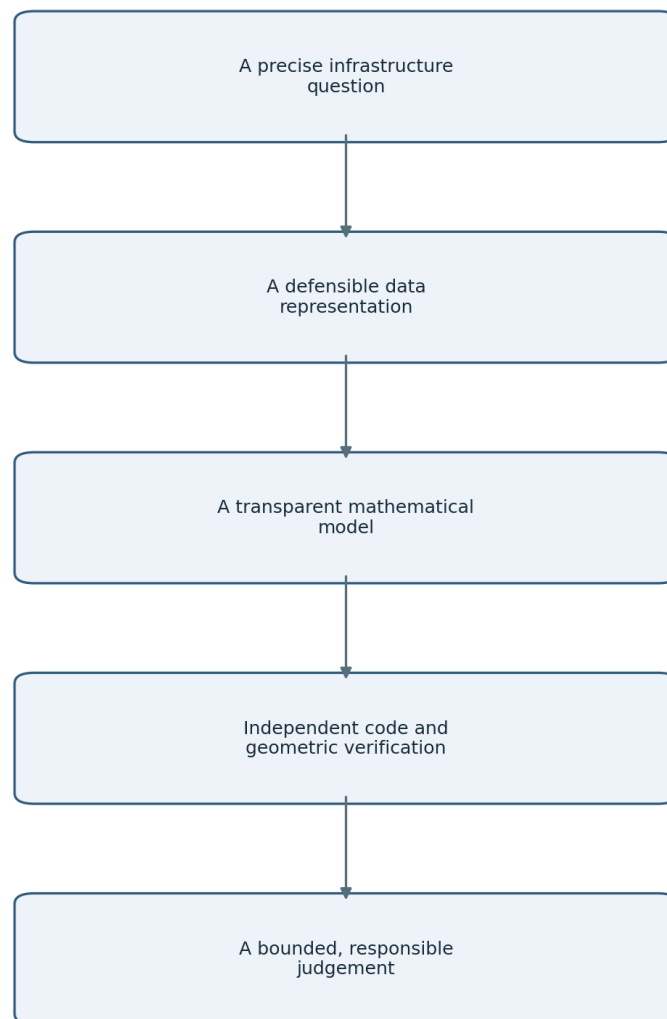
1. numerical conditioning and feature scaling;
2. QR and singular-value views of least squares;
3. probability models and maximum likelihood;
4. gradient descent built from the OLS gradient;
5. train/validation/test separation;
6. coefficient and prediction uncertainty; and
7. diagnostic plots and model revision.

The next week should not treat these as disconnected techniques. Each one answers a problem exposed by the model built here.

Final reflection

The intellectual achievement of Week 1 is not the formula $(X^T X)^{-1} X^T y$. It is the chain of reasoning around it:

You now know enough to be dangerous if you report coefficients carelessly—and enough to begin becoming useful if you preserve the chain from question to judgement.

**Figure 15:** Concept diagram